Overtone

Architecture of a Cross-Cultural Generative Music Engine in Rust

Simon Knight

Adelaide, Australia

March 2026 • Version 1.0.0

Abstract. Overtone is a terminal-based cross-cultural generative music engine written in Rust that produces full arrangements from declarative parameter configurations. The system combines manual oscillator synthesis, spectral sample analysis, and a hierarchical energy model to generate arrangement-aware music spanning Western electronic and Indian classical traditions in real time. This report presents the architecture of the synthesis pipeline, formalises the four-level energy model that drives pattern density and timbral evolution, describes the DSP chain (biquad filters, Schroeder reverb, ping-pong delay, sidechain compression), details the sample intelligence subsystem that classifies and blends audio samples with synthesised output, and documents the Ableton Live integration via OSC and Link. On an Apple M2 workstation the engine renders a 64-bar stereo track in 18 ms—approximately 470× real time—while maintaining 64-bit internal precision throughout the signal path.

Version	Date	Changes
1.0.0	March 2026	Initial release: 12 phases complete

Contents

List of Figures

List of Tables

1 Introduction

Psytrance is a subgenre of electronic dance music characterised by driving kick drums at 140–150 BPM, hypnotic bass lines built from filtered sawtooth oscillators, layered percussive textures, and carefully sculpted energy arcs that build tension and release over multi-minute arrangements. Producing a psytrance track in a conventional digital audio workstation (DAW) requires manually programming drum patterns, designing filter sweeps, tuning sidechain compression, and arranging dozens of clips across a timeline—a process that demands both musical knowledge and significant technical skill.

This report describes *Overtone*, a Rust-based generative music engine that automates the core production workflow. The user specifies high-level parameters—tempo, mood, key, energy curve—and the engine generates a complete stereo mix with kick, bass, hi-hat, clap, lead, and sample layers, all shaped by a hierarchical energy model that controls pattern density, filter modulation, and sample placement across the arrangement.

The system is designed around three principles:

1. **Energy-driven composition.** A four-level energy model (macro curve, meso breathing, micro groove, sub-channel gating) determines *what* plays *when* and *how loudly*, replacing manual arrangement with a declarative energy specification.
2. **Hybrid synthesis.** Each musical element can be rendered from manual oscillators, from classified audio samples, or from a configurable blend of both—adapting the timbral palette to the energy state of the arrangement.
3. **DAW integration.** MIDI export, real-time OSC clip pushing via AbletonOSC, and Ableton Link transport synchronisation allow the generator to function as a compositional front-end for professional production workflows.

The remainder of this report is structured as follows. **??** presents the system architecture and module decomposition. **??** formalises the energy model. **??** describes the synthesis engines (kick, bass, percussion, lead, and the additional drone, pad, plucked string, shruti box, and tom engines). **??** details the DSP processing chain, including the wavetable oscillator and PolyBLEP anti-aliasing. **??** covers the sample intelligence subsystem. **??** explains pattern generation and melody generation. **??** documents the eastern musical traditions layer, the Indian music theory modules (melakarta, thaat, rasa, murchhana), and the rhythmic device engines (tihai, korvai, layakari, nadai, tani, and the performance arc). **??** documents the Ableton integration layer, including MIDI import. **??** describes the terminal user interface. **??** presents the real-time streaming engine. **??** covers session and preset management. **??** evaluates rendering performance, and **??** concludes with future directions.

2 Theoretical Foundations

Overtone is built upon two distinct yet complementary musical traditions: Western electronic dance music (specifically psytrance) and Indian classical music (both Hindustani and Carnatic). This section details the theoretical frameworks from both traditions and explains how they are integrated within the engine.

2.1 Psytrance Theory: Rhythmic Drive and Energy Arcs

Psytrance is fundamentally a music of *texture* and *momentum*. Unlike functional harmony which relies on chord progressions, psytrance derives its narrative from the evolution of timbre over a static tonal center.

2.1.1 The Kick and Bass Relationship

The "engine room" of any psytrance track is the phase-coherent interplay between the kick drum and the bassline. The kick provides a steady quarter-note pulse (~140–150 BPM), while the bass occupies the 16th-note subdivisions between the kicks. The most common rhythmic relationship is the *offbeat* or *triplet* bass:

- **Offbeat:** Bass hits on every 16th note *except* the downbeat (e.g., [Kick, Bass, Bass, Bass]).
- **KB Syncopation:** Sidechain compression is critical here; the bass signal must "duck" (attenuate) when the kick triggers to prevent low-frequency masking and speaker clipping.

2.1.2 Energy-Driven Arrangement

Structure in psytrance is defined by *energy density*. Tracks typically follow a "Long Journey" arc:

1. **Intro:** Minimal percussion, atmospheric pads, drone elements.
2. **Build:** Progressive addition of percussion layers (hi-hats, claps) and increasing filter brightness.
3. **Drop/Peak:** Full rhythmic density, aggressive leads, maximum energy.
4. **Breakdown:** Sudden removal of rhythmic drive, focus on melodic motifs and atmospheric textures.

2.2 Indian Classical Theory: Raga and Tala

Indian classical music is built on two pillars: *Raga* (melodic mode) and *Tala* (rhythmic cycle).

2.2.1 Raga: The Melodic Framework

A raga is more than a scale; it is a "color" or "mood" with specific grammatical rules.

- **Aroh/Avroh:** The specific paths for ascending and descending. Some notes may be skipped in ascent (vakra) but included in descent.
- **Vadi and Samvadi:** The "King" and "Queen" notes. These are the most important degrees of the raga, where phrases typically land or resolve.
- **Pakad:** A characteristic melodic phrase that uniquely identifies the raga.

2.2.2 Tala: The Rhythmic Framework

Tala is a cyclical time system. A cycle (*avartan*) is divided into beats (*matras*), which are grouped into sections (*vibhags*).

- **Sam:** The first beat of the cycle, and the point of ultimate resolution.
- **Khali:** The "empty" beat, typically indicated by a wave of the hand, providing a structural counterpoint to the emphasized beats (*tali*).

2.2.3 Gamaka: Microtonal Ornamentation

Identity in raga music is carried by *gamakas*—ornaments that move between or around the notes.

- **Meend:** A continuous, expressive glide (portamento) between two notes.
- **Andolan:** A slow, gentle oscillation around a note.
- **Kampita:** A faster vibrato-like oscillation.

2.3 Cross-Cultural Synthesis in Overtone

Overtone bridges these worlds by mapping the grammatical rules of Indian classical music onto the 16-step grid and energy-driven arrangement of electronic music.

1. **Tala as Syncopation:** Rather than using complex time signatures, Overtone uses Tala cycles (e.g., Rupak 7, Jhaptal 10) to generate *accent patterns* across the 4/4 grid.

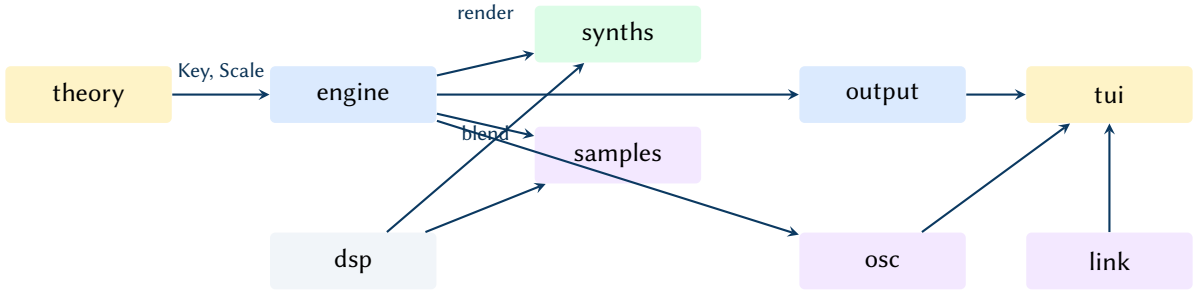

Figure 1. Module dependency graph. Colours indicate functional layers: gold for music theory, blue for core engine and output, green for synthesis, purple for integration and samples, grey for DSP primitives.

2. **Energy as Rasa:** The energy model is mapped to the *Navarasa* (nine aesthetic emotions). Low energy maps to *Shanta* (peace), while high energy maps to *Veera* (heroism) or *Raudra* (fury).
3. **Yoga-Flow Energy:** Arrangement arcs are modeled after *Yoga Asana* sequences—starting with grounding (Muladhara), building heat through the solar plexus (Manipura), and concluding in meditative dissolution (Sahasrara).

3 Architecture

Overtone is implemented as a single Rust binary (edition 2021) comprising nine top-level modules. The crate compiles to approximately 12 MB on macOS and has no runtime dependencies beyond the system audio driver (CoreAudio on macOS, ALSA on Linux).

3.1 Module Decomposition

?? illustrates the module dependency graph. Data flows left to right: the `theory` module provides musical primitives (keys, scales, moods) to the `engine`, which orchestrates pattern generation and calls into `synths` and `samples` for audio rendering. The `dsp` module provides shared signal processing primitives consumed by all synthesis code. The `output` module handles WAV export, MIDI export, and real-time playback. The `osc` and `link` modules manage Ableton integration, and the `tui` module provides the interactive terminal interface.

3.2 Key Data Structures

The system is organised around three central data structures:

- `GeneratorConfig` — the single source of truth for all generation parameters (BPM, key, mood, filter settings, energy curve, mute/solo state, pattern presets, humanisation level). Serialisable via `serde` for session persistence.
- `EnergyModel` — the hierarchical energy calculator that maps a bar number to per-element energy levels (??).
- `RenderResult` — the immutable output of a render pass, containing interleaved stereo samples, per-element stems, bar pattern metadata for visualisation, and MIDI export data.

Design Decision 1: Single Source of Truth

`GeneratorConfig` flows unmodified from CLI parsing through the TUI state into the `Generator`. Any parameter change in the TUI triggers a full re-render from the config, ensuring that the audio output always exactly reflects the visible parameter state. This design eliminates an entire class of state-synchronisation bugs at the cost of a re-render on every change—a cost that is negligible at 18 ms per 64-bar track (??).

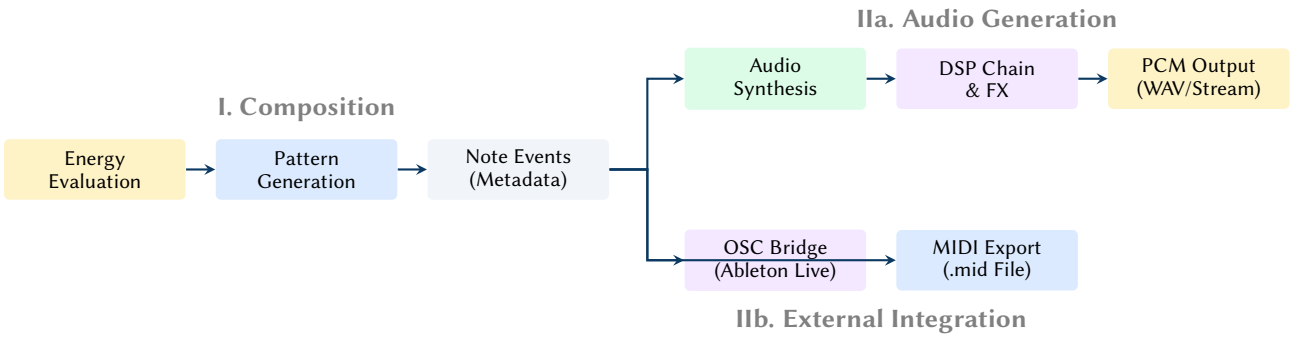

Figure 2. The decoupled pipeline architecture. The Composition stage (I) is deterministic and produces a structured event set. This set can then be rendered to high-precision audio (IIa) or exported directly to external environments (IIb).

3.3 The Two-Stage Pipeline: Composition vs. Generation

The system is architected as a two-stage pipeline where the **Compositional Logic** is strictly decoupled from the **Audio Generation**. This separation ensures that the engine can function as a standalone compositional front-end for DAWs (via MIDI/OSC) even when audio rendering is bypassed.

?? illustrates this split. The pipeline first resolves the musical structure into an immutable set of `NoteEvent` sequences before branching into one or more output targets.

1. **Stage I: Composition** — The `EnergyModel` is queried to determine the arrangement density and rhythmic character for each bar. The generators then produce `NoteEvent` sequences (pitch, velocity, timing) that exist purely as metadata.
2. **Stage II: Rendering (Optional)** — The note events are dispatched to either the internal synthesis engines (Stage IIa) or pushed to external targets like Ableton Live (Stage IIb).

> **Design Decision 2: Decoupled Output Targets**
>
> Because the `RenderResult` stores the note metadata separately from the audio buffers, the user can iterate on the composition (Stage I) while monitoring the TUI visualisation, and only commit to the computationally expensive audio rendering (Stage IIa) when necessary. Conversely, the engine can act as a pure "MIDI brain" for Ableton Live, bypassing the synthesis layer entirely to save CPU cycles.

4 The Energy Model

The energy model is the central organising abstraction of the generator. Rather than manually arranging patterns on a timeline, the user defines an energy curve and the model determines pattern density, filter modulation depth, sample selection, and element entry/exit timing. The model operates at four hierarchical levels.

4.1 Macro Curve

The macro curve defines the overall energy arc of the track as a piecewise-linear function $E_{\text{macro}}(b)$ over bar number b. It is specified as a sequence of control points $(b_i, e_i) \in [0, B] \times [0, 1]$ where B is the total number of bars:

$$E_{\text{macro}}(b) = e_i + (e_{i+1} - e_i) \cdot \frac{b - b_i}{b_{i+1} - b_i}, \quad b_i \le b < b_{i+1} \tag{1}$$

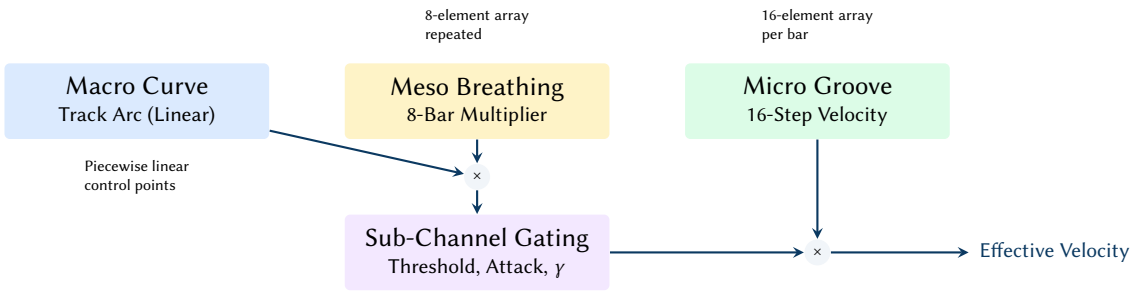

Figure 3. The four-level energy hierarchy. Macro and meso levels combine to drive the sub-channel gate, which is then modulated by the micro groove to produce the final step velocity.

Four named presets provide common arrangement shapes:

- **build_and_drop** — rises to 0.9 at 30%, drops to 0.3 at 35%, climbs to 1.0 at 70%, falls to 0.2 at end.
- **full_set** — monotonic rise from 0.2 to 1.0 at 75%, then a gentle decline to 0.7.
- **wave** — oscillating peaks at 0.6, 0.8, 0.9, 1.0 with valleys between, creating a rolling energy feel.
- **tension_release** — two sharp drops (0.7 → 0.2 at 30%, 0.9 → 0.3 at 60%) with recovery climbs, resolving to 1.0 at 90%.

4.2 Meso Breathing

The meso pattern imposes a phrase-level breathing rhythm as an 8-element multiplier array $M = [m_0, \ldots, m_7]$ that repeats every 8 bars. The default *breathing* pattern is:

$$M_{\text{breathing}} = [0.85,\ 0.90,\ 0.95,\ 1.0,\ 1.0,\ 0.95,\ 0.90,\ 0.85] \tag{2}$$

The combined energy at bar b is $E(b) = \text{clamp}(E_{\text{macro}}(b) \cdot M[b \bmod 8],\ 0,\ 1)$.

4.3 Micro Groove

The micro groove shapes velocity at the 16th-note level within each bar. A 16-element array $G = [g_0, \ldots, g_{15}]$ multiplies the velocity of each step, creating rhythmic accents. The *psy_groove* preset emphasises downbeats and offbeat drive:

$$G_{\text{psy}} = [1.0,\ 0.6,\ 0.75,\ 0.5,\ 0.9,\ 0.55,\ 0.8,\ 0.5,\ 0.95,\ 0.6,\ 0.75,\ 0.5,\ 0.85,\ 0.55,\ 0.8,\ 0.65] \tag{3}$$

4.4 Sub-Channel Gating

Each of the five musical elements (kick, bass, hi-hat, clap, lead) has an independent sub-channel that gates and shapes the element's response to the macro energy. A sub-channel is defined by four parameters:

The effective energy level for element i at bar b is:

$$E_i(b) = \begin{cases} 0 & \text{if } b < \text{onset}_i \text{ or } E(b) < \text{thresh}_i \\ \left(\min\left(1,\ \frac{b - b_{\text{active}}}{\text{attack}_i}\right) \cdot E(b)\right)^{\gamma_i} & \text{otherwise} \end{cases} \tag{4}$$

where b_{active} is the first bar at which both conditions (onset reached and threshold exceeded) are satisfied. The attack ramp fades the element in over attack_i bars after activation, and the response exponent γ_i shapes the energy-to-density transfer function.

Table 1. Default sub-channel parameters. The response curve exponent controls how aggressively the element responds to energy changes: < 1 gives a gentle ramp, > 1 gives a sharp threshold effect.

Element	Onset Bar	Threshold	Attack (bars)	Response γ
Kick	0	0.05	2	0.7
Bass	4	0.15	4	1.0
Hi-hat	8	0.25	4	1.2
Clap	8	0.35	2	1.5

Design Decision 3: Staggered Entry

The sub-channel onset and threshold parameters create a natural staggered entry: kick appears first (bar 0), bass enters at bar 4 once energy exceeds 0.15, and percussion layers arrive at bar 8. This mirrors the arrangement conventions of psytrance production, where elements are introduced progressively during the intro and build sections.

5 Synthesis Engines

The generator includes four dedicated synthesis engines, each implementing the specific timbral characteristics of its target element. All synthesis operates in `f64` precision; conversion to `f32` occurs only at the final output stage.

5.1 Kick Drum

The kick drum synthesiser produces up to 300 ms of audio per hit by summing three layers:

1. **Main body** — a sine oscillator with an exponential pitch envelope sweeping from $f_{\text{start}} \approx 150$ Hz to $f_{\text{end}} \approx 50$ Hz. The instantaneous frequency is:

$$f(t) = f_{\text{end}} + (f_{\text{start}} - f_{\text{end}}) \cdot e^{-\alpha_p t} \tag{5}$$

   where $\alpha_p \approx 35$ controls the pitch decay rate. The amplitude envelope is $a(t) = e^{-\alpha_a t}$ with $\alpha_a \approx 8$.

2. **Sub-harmonic** — a 25 Hz sine with a slower decay ($\alpha_{\text{sub}} \approx 2.5$) at 35% of the main body level. This extends the low-frequency sustain without muddying the transient.

3. **Click transient** — a short (~3 ms) burst of a high-frequency sine at ~3500 Hz with a linear fade-out. The click provides the initial attack snap that helps the kick cut through the mix.

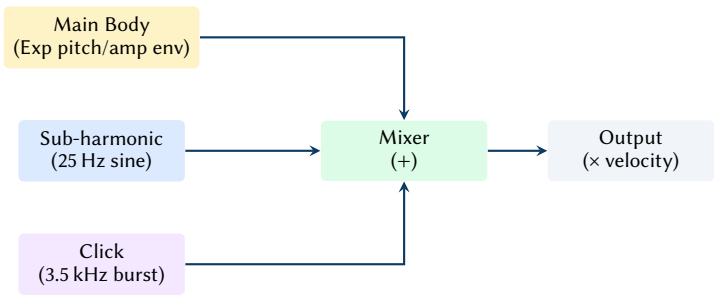

Figure 4. Kick drum synthesis model: parallel summation of main body, sub-harmonic, and click layers.

Phase Accumulation

All oscillators in the generator use phase accumulation ($\phi \mathrel{+}= f/f_s$) rather than direct computation ($\sin(2\pi f t)$). This is essential for the kick's pitch sweep: recomputing $\sin(2\pi f(t) \cdot t)$ with a time-varying $f(t)$ produces phase discontinuities that manifest as audible clicks at the sweep inflection points.

Design Decision 4: Critical Evaluation: Hardcoded Envelopes and Inflexibility

The kick synthesiser heavily relies on hardcoded constants to achieve its characteristic psytrance punch. For instance, the total duration of the kick is locked at 0.30 seconds (~13,230 samples at 44.1 kHz), and the sub-harmonic frequency is fixed to exactly 25 Hz.
While humanisation provides minor random variations, this rigid 3-layer architecture cannot accommodate drastically different kick styles, such as acoustic emulation, 808-style long decays, or hardstyle gabber kicks. The strict 0.30s cutoff abruptly zeroes out the engine, which can lead to low-frequency clicks if the envelope hasn't fully decayed. A more robust implementation would parameterize the decay curves and duration, making it a true general-purpose drum synthesiser rather than a single-trick pony.

5.1.1 Humanisation

The `humanize` parameter (0–1) scales per-hit random variation across all three layers. The humanised parameters and their variation ranges at full humanisation are:

- Start frequency: ±12 Hz
- End frequency: ±3 Hz
- Pitch decay rate: ±4
- Click frequency: ±400 Hz
- Click level: ±8%
- Sub level: ±7%

5.2 Bass Synthesiser

The bass engine renders one note at a time with a subtractive synthesis architecture: two detuned oscillators into a modulated biquad filter, followed by waveshaping and an ADSR amplitude envelope.

5.2.1 Oscillator Section

Two oscillators run simultaneously: a main oscillator at the note frequency f_0 and a detuned oscillator at $f_0 \cdot 2^{7/1200}$ (+7 cents). Each oscillator can be either a naive sawtooth (phase accumulation) or a wavetable oscillator scanning through a loaded wavetable. The mix ratio is 70% main, 30% detuned, creating a chorus-like thickening effect.

5.2.2 Filter Modulation

The biquad filter cutoff is modulated per-sample by three sources:

$$f_c(n) = f_{\text{base}} + \underbrace{E_{\text{filt}}(n) \cdot A_{\text{env}}}_{\text{envelope}} + \underbrace{L(n) \cdot D_{\text{lfo}}}_{\text{LFO}} \tag{6}$$

where $E_{\text{filt}}(n)$ is the filter envelope output (AD shape with configurable attack, decay, sustain, and amount in Hz), and $L(n)$ is the LFO output (unipolar, 0–1) with depth D_{lfo} in Hz. The filter's `set_params(`f_c`,` Q`)` method recomputes the RBJ biquad coefficients at every sample—a deliberate design choice that trades computational cost for artifact-free modulation.

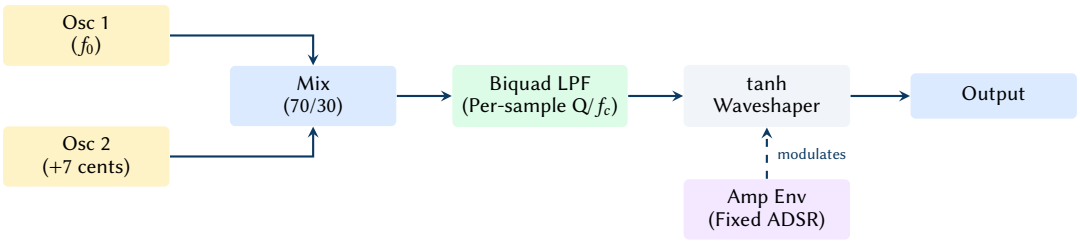

Figure 5. Bass synthesiser signal flow. Oscillators are mixed and passed through a biquad filter with per-sample coefficient updates, followed by soft clipping.

> **Design Decision 5: Critical Evaluation: Per-Sample Coefficient Updates & Hardcoding**
>
> While updating biquad coefficients per-sample prevents audible zipper noise during extremely fast filter sweeps (e.g., acid-style 0 ms attack envelopes), it incurs a substantial and often unnecessary computational penalty. At 44.1 kHz, computing trigonometric functions for coefficients on every sample dominates the bass rendering profile. In professional audio engines, block-rate updates (every 32–64 samples) combined with parameter interpolation are standard practice and achieve identical perceptual smoothness at a fraction of the cost.
> Furthermore, the amplitude envelope segments are strictly hardcoded (3 ms attack, 40 ms decay, 70% sustain, 30 ms release). While this guarantees a tight, psytrance-appropriate transient, it severely limits the engine's versatility for slower, evolving basslines in ambient or down-tempo contexts. The decision to hardcode these values rather than exposing them in the `GeneratorConfig` reflects a rigid design bias toward high-energy dance music, undermining the engine's stated goal of cross-genre adaptability.

Four filter envelope presets are provided:

Table 2. Filter envelope presets. The *amount* parameter specifies the peak cutoff offset in Hz above the base frequency.

Preset	Attack (ms)	Decay (ms)	Sustain	Amount (Hz)
Off	—	—	—	0
Acid	0	180	5%	2000
Slow Sweep	50	500	20%	1500
Pluck	0	80	0%	3000

5.2.3 Waveshaping and Envelope

The filtered signal is passed through a tanh waveshaper: $y = \tanh(1.3x)$, which adds odd harmonics and provides soft saturation. The amplitude envelope is a simple ADSR with fixed segments: 3 ms linear attack, 40 ms exponential decay to 70% sustain, sustain hold for the note duration, and 30 ms linear release.

5.3 Percussion

5.3.1 Hi-Hat

Hi-hats combine two sources: white noise filtered through a biquad HPF, and a metallic sine tone at approximately 7500 Hz. Closed hi-hats use a 7000 Hz HPF cutoff and fast amplitude decay (e^{-100t}, ~30 ms effective duration). Open hi-hats use a lower 6000 Hz cutoff and slower decay (e^{-15t}, ~150 ms).

5.3.2 Clap

The clap synthesiser produces a multi-tap noise envelope: three consecutive 10 ms bursts at 15 ms spacing (attenuating at 100%, 80%, 60%), followed by a 100 ms filtered noise tail with exponential decay. A 1000 Hz HPF removes low-frequency content. The multi-tap structure simulates the "multiple hands clapping" effect that gives the sound its characteristic scattered attack.

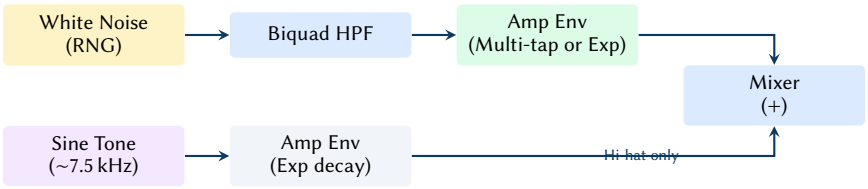

Figure 6. Percussion synthesis model. Hi-hats blend filtered noise with a metallic sine tone, while the clap relies entirely on a multi-tap noise envelope.

Design Decision 6: Critical Evaluation: Overly Simplistic Percussion Models

The percussion synthesiser reveals the most significant compromises in the engine's sound design. The hi-hat relies on a single metallic sine wave blended with filtered white noise, completely missing the complex, inharmonic interplay of multiple square wave oscillators (as seen in classic TR-808/909 architectures). This leads to a thin, somewhat static top-end.
Similarly, the clap synthesis relies on a rigidly hardcoded 3-tap delay structure (15 ms spacing). Real claps have stochastic tap spacing and density that varies per hit. By locking these parameters, the clap loses its organic feel and quickly becomes repetitive, necessitating the heavy use of the sample library (**??**) to mask the synthesiser's limitations.

5.4 Lead (FM Synthesis)

The lead synthesiser implements two-operator FM synthesis:

$$y(t) = A(t) \cdot \sin\left(2\pi f_c\, t + \underbrace{I \cdot E_{\text{fm}}(t) \cdot \sin(2\pi f_m\, t)}_{\text{modulator}}\right) \tag{7}$$

where f_c is the carrier frequency (note pitch), $f_m = f_c \cdot R$ is the modulator frequency (R is the FM ratio), I is the modulation index, and $E_{\text{fm}}(t) = e^{-5t}$ is a fast FM envelope that creates a time-varying timbre. The modulation index I and carrier amplitude decay are scaled by the macro energy level: higher energy produces brighter, longer lead tones.

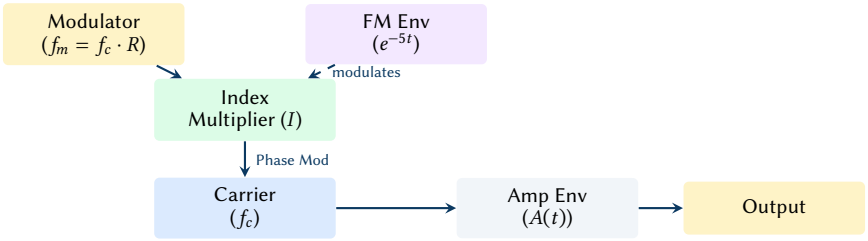

Figure 7. Lead synthesiser signal flow using 2-operator FM.

Design Decision 7: Critical Evaluation: Simplistic FM and Lack of Anti-Aliasing

The lead engine's implementation of FM synthesis is overly simplistic and suffers from significant structural limitations. Relying entirely on a hardcoded e^{-5t} decay envelope for the modulation index prevents the creation of brass-like swells or complex evolving textures that define modern FM patches.

More critically, the engine applies phase modulation directly to the carrier using a naive sine accumulator without any form of bandlimiting or oversampling. As the modulation index I scales up with macro energy, the generated sidebands quickly exceed the Nyquist frequency, folding back into the audible spectrum as harsh, inharmonic aliasing. While some degree of aliasing is characteristic of early digital synths (e.g., the Yamaha DX7), the absence of even rudimentary oversampling here degrades the high-frequency fidelity during peak energy sections, leading to a brittle and fatiguing sound.

5.5 Drone (Tanpura)

The drone synthesiser models a four-string tanpura, providing a continuous harmonic reference that fills sparse, low-energy sections of the arrangement.

5.5.1 Architecture

Four independent `DroneString` instances run in parallel, each consisting of two detuned oscillators (fundamental plus a cents-offset copy), three harmonics (fundamental at 100%, 2nd at 30%, 3rd at 15%), and a bandpass filter (220 Hz, $Q = 0.9$). The strings are tuned to Sa (low), Sa (low, slightly detuned), Pa or Ma (depending on the active raga's preferred drone third), and Sa (high octave).

Each string has a *pluck cycle*: a periodic re-excitation at intervals between 0.84 s and 1.18 s (staggered across strings to prevent phase alignment). The per-string envelope is:

$$a(t) = A_{\text{attack}}(t) \cdot e^{-4t/T_{\text{decay}}} \cdot (0.5 + 0.5 \sin(2\pi f_s t)) \tag{8}$$

where A_{attack} is a 2 ms linear ramp, $T_{\text{decay}} = 3$ s, and the sinusoidal swell adds ±4% amplitude variation that simulates the characteristic tanpura "throb".

5.5.2 Volume and Energy

The drone volume is *inversely* proportional to the macro energy level: $v = (0.08 + 0.12 \cdot (1 - E_{\text{macro}})) \cdot (c_v/0.15)$, where c_v is the user-configurable drone volume (default 0.15). This ensures the drone fills space during intros and breakdowns but recedes as percussion and bass dominate during peaks.

5.5.3 Jhala Layer

At high energy ($E > 0.85$) on eastern scales, the drone activates a *jhala* layer—rapid, rhythmically accented strikes on the upper strings. The jhala uses a 25% duty-cycle pulse wave with a four-hit accent cycle (strong 1.0, weak 0.55, weak 0.50, medium 0.75). The strike rate escalates with energy: eighth notes below 0.90, sixteenth notes from 0.90 to 0.95, thirty-second notes above 0.95.

5.6 Pad (Ensemble)

The pad synthesiser produces sustained, textured chords using a "super-saw" topology: three PolyBLEP sawtooth oscillators per voice, with up to four voices active simultaneously.

5.6.1 Voice Structure

Each `PadVoice` contains three oscillators at the note frequency, +1.5% detune, and −1.5% detune. Initial phases are spread at 0.0, 0.33, and 0.66 to prevent destructive cancellation on attack. A global LFO at 0.2 Hz modulates all pitches by ±0.5%, creating a slow drift.

The amplitude envelope uses long segments: 2.0 s linear attack, sustain hold, and 3.0 s linear release. The filter is a biquad LPF whose cutoff is modulated by both the LFO and the input amplitude:

$$f_c = 800 + L(t) \cdot 400 + |x_{\text{mix}}| \cdot 200 \quad \text{Hz} \tag{9}$$

5.6.2 Ensemble Chorus

A stereo chorus effect decorrelates the output using two delay taps per channel (15 ms left, 22 ms right) from 50 ms circular buffers. The mix is 70% dry, 30% wet.

When triggered from the streaming engine, the pad voices a minor chord voicing by default: root, minor third (+3 semitones), perfect fifth (+7), and minor seventh (+10).

5.7 Plucked String (Sitar/Veena/Sarod/Santoor)

The plucked string engine uses Karplus–Strong synthesis [**karplus1983**] with preset-specific extensions for four Indian string instruments.

5.7.1 Karplus–Strong Model

A delay line of length $N = f_s/f_0$ samples is excited with a short noise burst (lowpass-filtered), then fed back through a one-pole averaging filter:

$$y[n] = (1 - d) \cdot \frac{x[n] + x[n-1]}{2} + d \cdot y[n-1] \tag{10}$$

where d is the damping coefficient and the feedback gain controls the decay time. **??** lists the preset parameters.

Table 3. Plucked string preset parameters.

Preset	Feedback	Damping	Jawari	Sympathetic	Resonance
Sitar	0.997	0.12	Yes (0.85)	6 strings	0.06
Veena	0.995	0.25	No	4 strings	0.05
Sarod	0.993	0.10	No	5 strings	0.04
Santoor	0.986	0.08	No	0	—

5.7.2 Jawari (Bridge Buzz)

The sitar preset includes a *jawari* filter that models the curved bridge responsible for the instrument's characteristic nasal buzz. Below a buzz threshold, the signal is reshaped: $y = \text{sign}(x) \cdot |x|^{0.65}$, emphasising upper partials at low amplitudes. The bridge curve parameter (0.85) controls the wet/dry blend.

5.7.3 Sympathetic Strings

Up to six sympathetic strings (Karplus–Strong instances with feedback 0.998 and damping 0.28) resonate in response to the main string output, scaled by a resonance coefficient. The sympathetic halo is mixed at 25% of main amplitude.

5.8 Shruti Box (Electronic Harmonium)

The shruti box synthesiser models an Indian harmonium drone with bellows-driven amplitude modulation.

5.8.1 Oscillator Bank

Four oscillators generate harmonically rich reed tones using six-harmonic additive synthesis: $y = 0.25 \sum_{k=1}^{6} h_k \sin(2\pi k f \phi)$, where the default harmonic amplitudes are $h = [1.0, 0.3, 0.7, 0.15, 0.5, 0.08]$. The number of active oscillators depends on the mode: Minimal (1), Standard (2), Devotional (2), Full (4).

5.8.2 Bellows Envelope

An asymmetric triangle LFO at 0.25 Hz simulates the pump action: $a_{\text{bellows}}(t) = 1 - d + d \cdot T(t)$, where $d = 0.15$ is the modulation depth and $T(t)$ is an asymmetric triangle (40% rise, 60% fall) that models fast push and slow pull.

The output passes through per-channel bandpass filters (520 Hz, $Q = 0.9$) with the right channel offset by +3% for stereo width. Filter cutoff modulates with energy: $f_c = 420 + 280 \cdot E$ Hz.

5.9 Tom Drum

The tom drum synthesiser renders deep, resonant drum hits with three distinct articulations modelled after tabla strokes:

- **Dha** (open resonant) — pitch sweep from $2.5 f_0$ to f_0 (14 ms decay), detuned second oscillator, second harmonic at −12 dB, 2 ms noise burst. 500 ms duration, LPF at $4 f_0$.
- **Tin** (closed sharp) — higher pitch ($f_0 \cdot 2^{7/12} \cdot 1.3$), fast decay (22 ms), metallic ring partial at $2.7 f_0$ (−9.5 dB), BPF at $8 f_0$. 80 ms duration.
- **Ghe** (sliding bass) — pitch sweep from $f_0 \cdot 2^{-5/12}$ to $0.6 f_0$ (10 ms decay), 1 ms noise burst, LPF at 400 Hz. 300 ms duration.

All three use exponential pitch envelopes: $f(t) = f_{\text{end}} + (f_{\text{start}} - f_{\text{end}}) \cdot e^{-\alpha_p t}$. Per-hit randomisation is applied to detune ratio, noise level, and amplitude decay rate.

6 DSP Processing Chain

The `dsp` module provides shared signal processing primitives used by all synthesis engines and the master effects chain. **??** illustrates the signal flow through the effects stage.

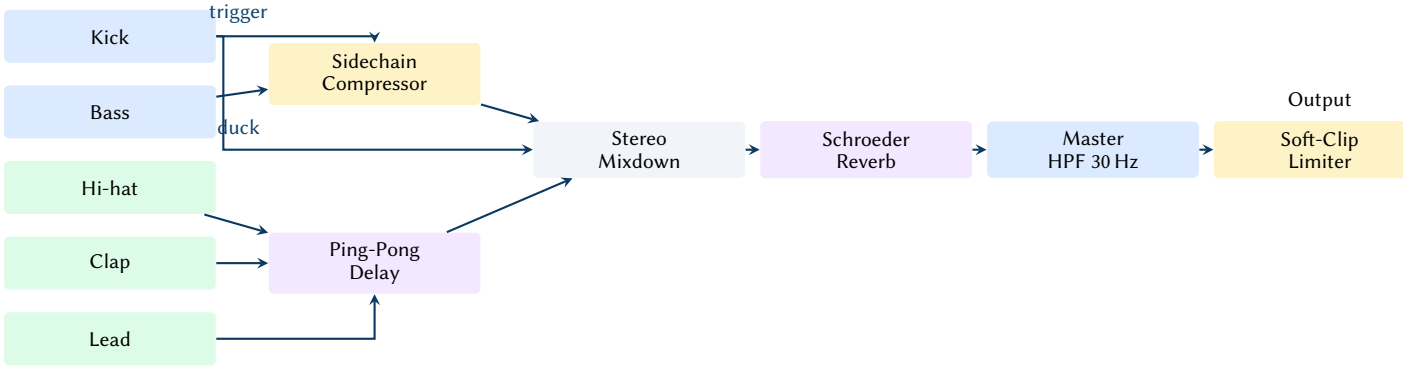

Figure 8. Master effects signal flow. The kick triggers the sidechain compressor which ducks the bass. Hi-hat, clap, and lead pass through the ping-pong delay before joining the stereo bus.

6.1 Biquad Filter

The `Biquad` struct implements a second-order IIR filter using the RBJ Audio EQ Cookbook [**rbj**] formulas. It supports four filter types: low-pass, high-pass, band-pass, and notch. The filter processes samples using Direct Form II Transposed:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2] \tag{11}$$

Stability is maintained by clamping the cutoff frequency to $[20,\ 0.45 f_s]$ Hz and the Q factor to $[0.5, 20]$. The upper cutoff clamp at $0.45 f_s$ (rather than $0.5 f_s$) prevents numerical instability as ω_0 approaches π.

6.2 Sidechain Compression

The sidechain compressor ducks the bass signal using the kick's amplitude envelope as a control signal:

$$g_{\text{sc}}[n] = 1 - d \cdot \min(3 \cdot s[n],\ 1) \tag{12}$$

where $d = 0.75$ is the duck depth and $s[n]$ is the kick envelope follower with instantaneous attack and exponential release:

$$s[n] = \begin{cases} |x_{\text{kick}}[n]| & \text{if } |x_{\text{kick}}[n]| > s[n-1] \\ s[n-1] \cdot \alpha_r & \text{otherwise} \end{cases} \tag{13}$$

where $\alpha_r = e^{-1/(0.12 f_s)}$ gives a 120 ms release time constant.

6.3 Ping-Pong Delay

The `PingPongDelay` uses two circular buffers (left and right) with cross-feedback routing and a one-pole IIR low-pass filter in each feedback path for analog-style warmth:

$$\text{buf}_L[w] = x_L + \text{fb}_R \cdot k \tag{14}$$

$$\text{buf}_R[w] = x_R + \text{fb}_L \cdot k \tag{15}$$

where k is the feedback coefficient (clamped to 0.95 maximum) and w is the write position. The delay time is tempo-synchronised: a dotted 16th note at the current BPM, computed as $t_d = 0.375 \cdot 60/\text{BPM}$ seconds.

Default parameters: 35% feedback, 60% damping coefficient, 20% wet mix.

6.4 Schroeder Reverb

The `StereoReverb` implements the classic Schroeder topology [**schroeder1962**] with decorrelated channels:

1. **Four parallel comb filters per channel** (eight total). Delay lengths at 44.1 kHz are chosen to be mutually prime to prevent flutter echo: left channel uses $\{1116, 1188, 1277, 1356\}$ samples; right channel uses $\{1139, 1211, 1300, 1379\}$. A one-pole low-pass filter in each feedback path provides frequency-dependent damping.
2. **Two series allpass filters per channel.** Delays: left $\{556, 441\}$, right $\{579, 464\}$. The allpass coefficient $g = 0.5$.
3. **Wet/dry mix** at 12% wet.

Default parameters: decay coefficient 0.4, damping 0.7, giving an approximate RT60 of 0.6 s.

6.5 Master Processing

Two final processing stages are applied to the stereo mix:

- **30 Hz high-pass filter** — a second-order Butterworth (biquad HPF, $Q = 0.707$) removes subsonic energy accumulated from the kick's sub-harmonic layer and bass oscillator DC drift.
- **Soft-clip limiter** — normalises to −0.5 dBFS peak, then applies a tanh soft clipper with a knee at 0.9:

$$y = \begin{cases} x & |x| \leq 0.9 \\ \text{sign}(x) \cdot \left(0.9 + 0.1 \cdot \tanh\left(\frac{|x|-0.9}{0.1}\right)\right) & |x| > 0.9 \end{cases} \tag{16}$$

6.6 Wavetable Oscillator

The `Wavetable` structure stores multi-cycle waveforms as a flat `Vec<f32>` with a fixed cycle length (default 2048 samples). The `WavetableOscillator` scans through cycles using two parameters: `phase` (position within a single cycle, 0–1) and `cycle_pos` (morphing position across cycles, 0–1).

6.6.1 Cross-Cycle Interpolation

Sample lookup uses bilinear interpolation across both the cycle dimension and the phase dimension:

$$y(\text{cp}, \phi) = (1 - \alpha) \cdot S(c_0, \phi) + \alpha \cdot S(c_1, \phi) \tag{17}$$

where $c_0 = \lfloor \text{cp} \cdot (N_c - 1) \rfloor$, $c_1 = c_0 + 1$, α is the fractional cycle position, and $S(c, \phi)$ performs linear interpolation within a single cycle between adjacent samples. This produces smooth timbral morphing as `cycle_pos` sweeps from the first waveform shape to the last.

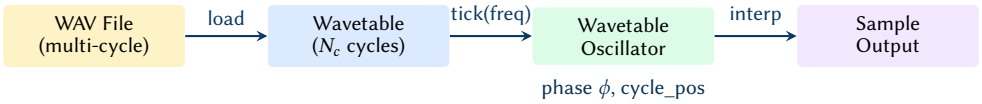

Figure 9. Wavetable oscillator data flow: WAV loading, cycle storage, and bilinear interpolation at playback.

6.6.2 Wavetable Library

The `WavetableLibrary` maintains a `HashMap` of named wavetables. Two synthetic defaults are generated at initialisation: a sawtooth (2048-sample linear ramp −1 to +1) and a square (50% duty pulse). Additional wavetables are loaded from a user-specified directory by scanning for WAV files and splitting each into 2048-sample cycles.

6.7 PolyBLEP Anti-Aliasing

The `PolyblepOscillator` reduces digital aliasing on sawtooth and square waveforms by applying a second-order polynomial correction at phase discontinuities.

6.7.1 Correction Function

The PolyBLEP residual is a piecewise quadratic applied near $t = 0$ and $t = 1$ in the phase domain:

$$\text{polyblep}(t, \Delta t) = \begin{cases} \frac{t}{\Delta t}\left(2 - \frac{t}{\Delta t}\right) - 1 & t < \Delta t \\ \left(\frac{t-1}{\Delta t}\right)^2 + 2\frac{t-1}{\Delta t} + 1 & t > 1 - \Delta t \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

where $\Delta t = f / f_s$ is the phase increment per sample.

6.7.2 Bandlimited Waveforms

The corrected sawtooth subtracts the PolyBLEP residual from the naive ramp: $y_{\text{saw}} = (2t - 1) - \text{polyblep}(t, \Delta t)$. The square applies the correction twice—at $t = 0$ and at $t = 0.5$—to smooth both transitions:

$$y_{\text{sq}} = \text{sign}(0.5 - t) + \text{polyblep}(t, \Delta t) - \text{polyblep}((t + 0.5) \bmod 1, \Delta t) \tag{19}$$

The PolyBLEP oscillator is currently used in the pad synthesiser (**??**) for alias-free detuned sawtooths, and is available for future application to lead and bass oscillators.

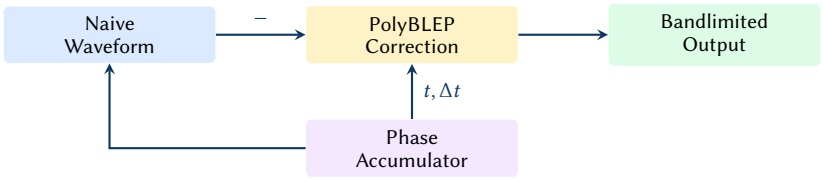

Figure 10. PolyBLEP anti-aliasing: the correction term is subtracted from the naive waveform at phase discontinuities.

7 Sample Intelligence

The sample intelligence subsystem analyses, classifies, and blends audio samples with the synthesised output. It operates in four stages: spectral analysis, rule-based classification, cached profile storage, and energy-aware placement.

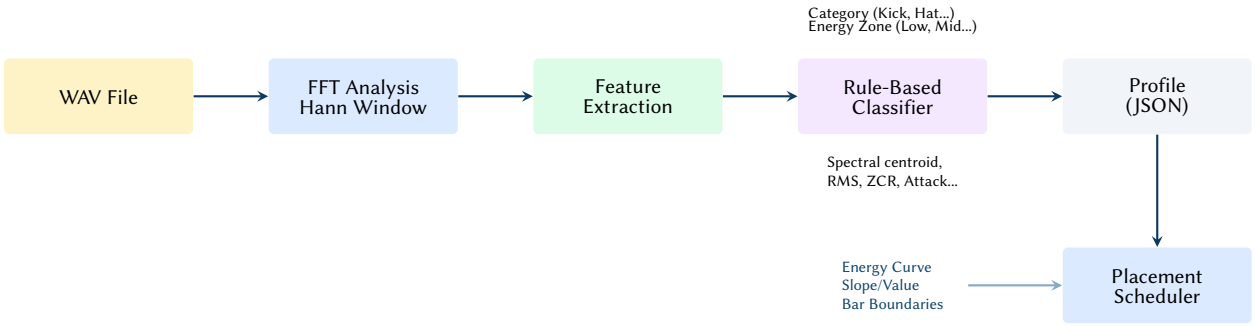

Figure 11. The sample intelligence pipeline. Raw audio is processed via spectral analysis to extract acoustic features, classified into categories and energy zones, and stored as JSON profiles. The scheduler then uses macro energy state to place samples across the arrangement.

7.1 Feature Extraction

The `analyzer` module extracts seven acoustic features from each loaded WAV file using `rustfft` [**rustfft**] for spectral analysis:

7.2 Classification

The `classifier` module applies a rule-based decision tree to assign each sample to one of nine categories: `Kick`, `Snare`, `Hat`, `Clap`, `Bass`, `Fx`, `Riser`, `Texture`, `Vocal`.

```
fn classify(profile: &SampleProfile, hint: Option<SampleCategory>)
    -> SampleCategory
{
    let p = profile;
    if p.spectral_centroid < 300.0
        && p.zcr < 0.05
```

Table 4. Extracted acoustic features per sample.

Feature	Unit	Method
Spectral centroid	Hz	Energy-weighted mean frequency from Hann-windowed FFT ($\bar{f} = \sum f_i \lvert X_i \rvert / \sum \lvert X_i \rvert$)
RMS level	0–1	Root mean square amplitude
Zero crossing rate	0–1	Sign change count normalised by sample length
Attack time	ms	Time to reach 90% of peak RMS (5 ms analysis frames)
Tonal score	0–1	Normalised autocorrelation peak over lags 1 to $N/2$
Amplitude slope	—	Linear regression slope of per-frame RMS (positive = riser)
Brightness	0–1	Normalised centroid: $\min(1, \text{centroid}/10000)$

```
        && p.attack_ms < 50.0
    {
        return SampleCategory::Kick;
    }
    if p.duration_ms > 2000.0 && p.amplitude_slope > 0.0 {
        return SampleCategory::Riser;
    }
    if p.spectral_centroid > 3000.0 && p.zcr > 0.15 {
        return SampleCategory::Hat;
    }
    // ... additional rules omitted for brevity
    hint.unwrap_or(SampleCategory::Fx)
}
```

When the rule-based classifier cannot determine a category, it falls back to a directory-name hint (e.g. a sample in a `kicks/` subdirectory is classified as `Kick`).

7.3 Profile Caching

Computed profiles are stored as sidecar JSON files (`<stem>.sample.json`) alongside the source WAV. Cache validity is checked against the source file's modification timestamp and a schema version counter, ensuring that profiles are automatically recomputed when the analyser is updated.

7.4 Energy-Aware Placement

The `PlacementScheduler` pre-computes per-bar placement decisions for non-drum sample categories (FX hits, risers, textures, fills, vocals) based on the macro energy curve's value and slope:

- **Risers** — placed at 4-bar boundaries when the energy slope exceeds 0.15, with length chosen from $\{4, 8, 16\}$ bars to match the estimated climb duration.
- **FX hits** — at 4-bar boundaries when energy > 0.80; additionally at 2-bar boundaries when energy > 0.90.
- **Textures** — on energy plateaus ($|\text{slope}| < 0.05$) above 0.30 energy, at 4/8/16-bar intervals depending on density.

- **Fills** — at 4-bar boundaries above threshold, always at 8 and 16-bar boundaries.
- **Vocals** — every 16 bars, outside the 0.4–0.8 energy range (appearing in intros and peaks but not mid-energy builds).

7.5 Synth/Sample Blending

Each element's output can be pure synthesis, pure sample playback, or a weighted blend. The default blend ratio is computed from the macro energy level:

$$r_{\text{blend}} = \text{clamp}(0.8 \cdot E_{\text{macro}},\ 0,\ 0.8) \tag{20}$$

At low energy, synthesis dominates (clean, controlled sound); at high energy, samples are blended in (adding the organic variation and complexity of real recordings). Per-element blend overrides are available in the TUI.

8 Pattern Generation

All pattern generators take an `EnergyState` (the combined output of the energy model for the current bar) and produce a `BarPattern` containing a vector of `NoteEvent` structs:

```
pub struct NoteEvent {
    pub step: u8,              // 0..15 (16th note grid)
    pub velocity: f64,         // 0.0..1.0
    pub pitch_hz: f64,         // fundamental frequency
    pub duration_steps: u8,    // note length in 16ths
    pub timing_offset_samples: i32,  // micro-timing humanisation
}
```

8.1 Energy-Driven Density

Pattern density scales with the element's energy level. For bass, the number of active steps per bar is:

$$N_{\text{steps}} = \text{round}(4 + E_{\text{bass}} \cdot 12) \tag{21}$$

yielding 4 steps at zero energy (quarter notes) and 16 steps at full energy (continuous 16th notes).

8.2 Presets and Variation

Each element has multiple pattern presets that define distinct rhythmic characters. **??** summarises the kick presets as a representative example.

Table 5. Kick pattern presets. Each preset defines placement rules that are further modulated by the energy level.

Preset	Description
Default	Beats 1 and 3 always; beats 2 and 4 if energy > 0.3; ghost 16ths if > 0.7; stochastic pickups and flams
Minimal	Beats 1 and 3 only
Driving	All four beats plus probabilistic 16th-note fills
Breakbeat	Syncopated placement at steps $\{0, 3, 6, 10, 14\}$

8.3 Micro-Timing Humanisation

Three layers of humanisation are applied, all scaled by the global `humanize` parameter (0–1):

1. **Velocity jitter** — ±12%, with softer hits varying more: jitter $= 1 \pm 0.12 \cdot (1 - v \cdot 0.5)$.
2. **Micro-timing** — downbeats are displaced by ±1 ms; offbeats by ±3.5 ms. At 44.1 kHz, 3.5 ms is approximately 154 samples.
3. **Timbral variation** (kick only) — per-hit variation in pitch envelope, click frequency, and sub-harmonic level (**??**).

> **MIDI Preservation**
>
> Micro-timing offsets are preserved in MIDI export. At 960 PPQ, one tick is approximately 0.44 ms at 142 BPM, so a ±3.5 ms offset maps to ±8 MIDI ticks—sufficient resolution to reproduce the humanisation feel in a DAW.

8.4 Psytrance Pattern Logic: The Psy_Groove

The quintessential psytrance rhythm is a 16th-note subdivision where the kick drum occupies the first 16th note of each quarter-note beat (the "four-to-the-floor" foundation). The bassline interacts with this grid to create the driving momentum characteristic of the genre. Overtone implements several variations of this relationship:

- **Offbeat (1/1):** The most foundational pattern, where a single bass hit occurs on the 16th note immediately following each kick.
- **Rolling (3/1):** Three consecutive bass hits follow each kick, filling the remaining 16th-note slots in the beat. This creates a relentless, "rolling" forward motion.
- **Galloping (1/2):** A long kick hit followed by two rapid bass hits, creating a "gallop" feel (e.g., [Kick, *gap*, Bass, Bass]).
- **Syncopated (16th):** Bass hits are placed on syncopated 16th-note positions (e.g., steps 3, 6, 9, 12), creating cross-rhythmic tension against the steady kick pulse.

These patterns are implemented in `engine/patterns.rs` using a step-weighting system. At low energy, the engine prefers simple offbeat patterns; as energy increases, it transitions into rolling or syncopated variations to heighten the driving feel.

8.5 Raga-Aware Melody Generation

The `RagaMelodyGenerator` produces bar-length melodic patterns for lead synthesis when an eastern scale is active. It respects the raga's aroh (ascending path) and avroh (descending path) and weights pitch selection toward the vadi (primary important tone) and samvadi (secondary important tone).

8.5.1 Density and Direction

Note density scales with macro energy: 3 notes per bar below 45% energy, 4 notes between 45% and 75%, and 8 notes above 75%. The generator maintains a traversal direction (ascending or descending) that flips with 25% probability each bar, producing melodic contour variety. Pitch candidates are drawn from the direction-appropriate path when the scale defines explicit aroh/avroh sequences.

8.5.2 Pitch Selection

At low energy (< 0.45), pitches are drawn uniformly from the scale's stable tones (root, preferred drone third, perfect fifth). At higher energy, a weighted random selection favours the vadi (40% probability) and samvadi (25%), with the remaining probability spread across all allowed degrees. Octave selection is one octave above the root by default, with a 15% chance of jumping to two octaves above 75% energy.

8.5.3 Ornamentation

Each generated note may receive a `GamakaSpec` ornament based on the raga's per-pitch gamaka rules:

- **Andolan** — slow oscillation (0.9 Hz, 20–60 cents depth).
- **Kampita** — sharp vibrato (4.0 Hz, 8–25 cents depth).
- **Meend** — pitch glide from the previous note (90 ms slide), triggered when the interval is ≤ 4 semitones and energy exceeds 45%.
- **Kan-swar** — grace note approach (± 1 semitone, 20 ms), triggered at 15% probability above 65% energy.

Note duration decreases with energy: 3 steps (held) below 45%, 2 steps at moderate energy, 1 step (rapid-fire) above 75%. Velocity is accented on the first note of each bar (0.95) and jittered by $\pm 8\%$ on subsequent notes.

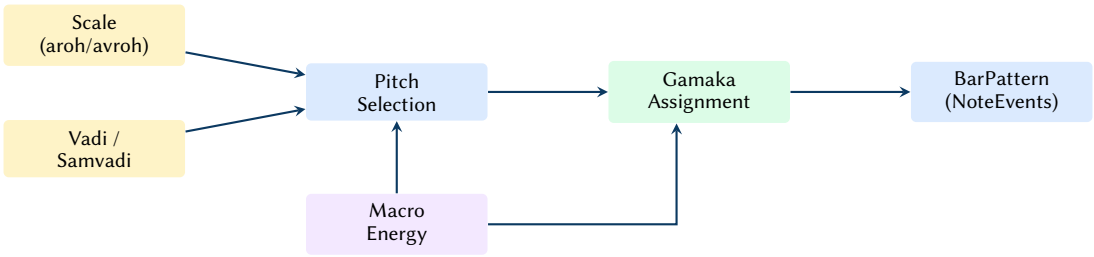

Figure 12. Raga melody generation: scale constraints and vadi/samvadi weighting feed pitch selection, energy controls density and ornamentation depth, and gamaka rules produce per-note ornaments.

8.6 Arrangement Sections

The `arrangement` module classifies each bar into one of four structural sections—Intro, Build, Peak, or Breakdown—using a heuristic decision tree over the macro energy curve's value, slope, and song progress:

Table 6. Arrangement section detection heuristics. Progress is measured as bar/total_bars.

Section	Conditions
Intro	Progress $< 12\%$ and energy < 0.45; or energy < 0.35 at any point
Build	Slope > 0.02 (rising energy); or default for mid-range energy
Peak	Energy ≥ 0.85
Breakdown	Progress $> 30\%$, slope < -0.04, energy < 0.60; or energy < 0.35 after 30% progress

These section labels are used by MIDI export, Ableton scene mapping, and TUI status display.

9 Eastern Musical Traditions in Electronic Music

While the initial design targeted classic Western modes and 16-step psytrance syncopation, the architecture generalises cleanly to other musical traditions. This project includes raga-inspired scales and tala-inspired accent cycles that can be applied without changing the global time signature.

9.1 Specific Raga Profiles and Their Applications

While all 72 melakarta scales are available, the generator provides several hand-crafted raga-inspired profiles used in the mood presets:

- **Bhairav (Hindustani):** Characterized by flattened 2nd (Re) and 6th (Dha) degrees. It creates a solemn, dawn-like character, making it effective for meditative intros and "Dark Phrygian" style psytrance.
- **Ahir Bhairav (Hindustani):** Combines the Bhairav lower tetrachord with a more major-sounding upper tetrachord. It is bright yet devotional, ideal for progressive melodies and "Goa Sunrise" themes.
- **Todi (Hindustani):** An intensely chromatic and tense raga, used to create maximum psychedelic tension during energy build-ups.
- **Malkauns (Hindustani):** A pentatonic (five-note) raga of deep introspection and late-night calm. Its sparse structure is ideal for ambient drones and "Samay" midnight presets.
- **Charukesi (Carnatic):** A lyrical and highly emotional raga, used for progressive themes that require a balance of melodic warmth and mystic color.

9.2 Tala Accent Cycles and Rhythmic Structures

Indian classical rhythm (*tala*) is modeled as a cyclic system of accentuation. Rather than using complex time signatures, Overtone applies tala-derived velocity multipliers onto the 16-step grid:

- **Rupak (7 beats):** Often interpreted as $[3, 2, 2]$. Its syncopation against the 16-step grid creates an evolving cross-rhythmic pattern that resolves every 7 beats.
- **Jhaptal (10 beats):** interpreted as $[2, 3, 2, 3]$. It creates a steady yet complex groove suitable for mid-tempo progressive tracks.
- **Ektal (12 beats):** Often used in fast-tempo compositions, providing a high-density rhythmic drive.

9.3 Indian Classical Structural Devices

Beyond interval sets, the generator implements several structural devices to maintain authentic formal development:

- **Aroh and Avroh:** Many ragas have different paths for ascending (*aroh*) and descending (*avroh*). The melodic generator maintains a directional traversal state and selects pitch candidates based on these paths.
- **Vadi and Samvadi:** The "King" and "Queen" notes are emphasized through a weighting bias in the pitch selection algorithm, ensuring the melodic center of the raga is properly established.
- **Gamakas (Ornaments):** The system can attach `GamakaSpec` metadata to notes (meend, andolan, kampita), which the internal synthesis engines interpret as per-sample frequency or amplitude modulations.
- **Tihai:** A cadential device where a phrase is repeated three times to resolve on the downbeat (*sam*). The engine can generate a tihai as a final bar-override before a section boundary (e.g., from Build to Peak).
- **Sangati:** Repetition with progressive elaboration. Each time a phrase is repeated across multiple bars, the engine can slightly increase the melodic density or add ornaments to mimic the live development of a composition.

- **Jugalbandi:** A call-and-response duet mode where two synthesis layers (e.g., Lead and FM String) exchange phrases, with the "respondent" layer varying the "caller's" motif.

9.4 Yoga Flow and Rasa-to-Energy Mapping

Overtone extends the traditional Western arrangement by mapping the energy model to the aesthetic and philosophical frameworks of Indian music:

1. **Navarasa:** The "Nine Emotions" framework. The generator can bias parameters based on a selected rasa:
   - **Shanta (Peace):** Lower energy, slow breath LFO, drone-dominant.
   - **Veera (Heroic):** Rising energy, sharp attacks, Ahir Bhairav scale.
   - **Adbhuta (Wonder):** Moderate energy, high filter resonance, random walk arpeggios.
2. **Chakra Architecture:** Arrangement arcs are modeled after *Yoga Asana* sequences. The "Yoga Flow" energy curve moves from Muladhara (grounding/intro) to Manipura (strength/peak) and then to Sahasrara (transcendence/cool-down).

9.5 Motif Foreshadowing via Sub-Channel Preview

Sub-channel gating includes a preview window below the main activation threshold. In this region elements can appear as sparse "ghost" hits, hinting at future motifs before the full density arrives. The result is a stronger sense of narrative continuity: motifs are introduced, developed, and then brought into the foreground.

9.6 Drone, Ornamentation, and Cadential Devices

Beyond interval sets and accent cycles, Indian classical identity is often carried by a small set of structural devices:

- **Tanpura drone** — a continuous pitch reference (Sa–Pa–Sa) that anchors intonation and fills sparse sections. The engine can render a four-string drone layer tuned to the active key and mixed inversely with macro energy.
- **Aroh/avroh constraints** — ragas are not only scales; they encode preferred ascending and descending motion. A raga-aware melody generator uses directional step sets and avoids stepwise motion that contradicts the raga's character.
- **Vadi/samvadi weighting** — note hierarchy influences phrase landing points and perceived emotional center. The melodic generator can weight pitch selection toward the vadi (primary) and samvadi (secondary) degrees.
- **Gamaka** — ornaments such as meend (glide), kan-swar (grace approach), and andolan/kampita (oscillation) provide micro- expression that distinguishes ragas even in 12-TET approximations.
- **Tabla bols and articulation** — percussive syllables (e.g., Dha, Tin, Ghe) encode stroke families and resonance. The tom layer can attach articulation metadata to events and render different envelopes/spectra per stroke.
- **Tihai and korvai** — cadential patterns that resolve on sam (downbeat). These devices can be inserted at section boundaries to increase formal coherence and create a clear sense of arrival.

9.7 Time, Notation, and Breath

Two additional ideas extend the system beyond purely pattern-driven EDM generation:

- **Samay** — time-of-day associations can bias raga selection toward traditional prahar groupings.
- **Sargam + bols in the TUI** — displaying Sa/Re/Ga/Ma or bol initials in the step grid provides a musically meaningful view of eastern-mode patterns.

- **Breath-synced modulation** — a slow asymmetric LFO can modulate global dynamics and timbre (filter cutoff, wetness), and reduce high-frequency density during exhale to support meditative outputs.

9.8 Melakarta Raga Classification

The `theory/melakarta` module implements the complete Carnatic 72-melakarta system as a generative lookup table. Each melakarta is defined by its canonical number (1–72), name, seven-note interval set, chakra group, and tonal character description.

9.8.1 Combinatorial Generation

The 72 melakartas are generated combinatorially from three independent choices:

1. **Ma (madhyama)** — 2 options: shuddha (5 semitones) or prati (6 semitones).
2. **Ri–Ga pairs** — 6 combinations from $\{(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)\}$ semitones.
3. **Da–Ni pairs** — 6 combinations from $\{(8,9),(8,10),(8,11),(9,10),(9,11),(10,11)\}$ semitones.

The total $2 \times 6 \times 6 = 72$ ragas are indexed into 12 chakras (6 ragas per chakra) via chakra $= \lfloor (n-1)/6 \rfloor$. Nine notable ragas with hardcoded personality descriptions are highlighted for UI display: Kanakangi ("mysterious, tense"), Mayamalavagowla ("grand, devotional"), Natabhairavi ("pathos"), Kharaharapriya ("versatile, deep"), Harikambhoji ("bright, joyful"), Dheerasankarabharanam ("complete, bright"), Chalanata ("alien, otherworldly"), Kamavardhini ("intense, dramatic"), and Mechakalyani ("serene, luminous").

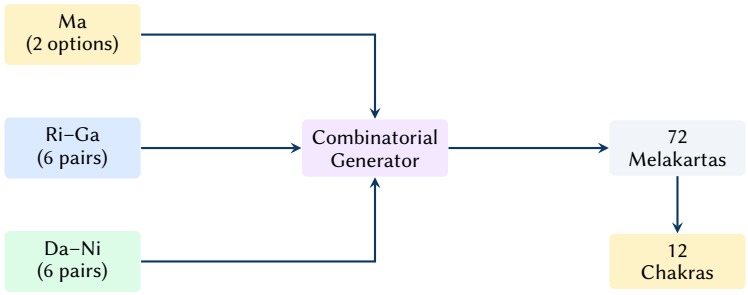

Figure 13. Melakarta generation: three independent pitch choices yield 72 canonical ragas grouped into 12 chakras.

9.9 Thaat Classification (Hindustani)

The `theory/thaat` module defines the 10 canonical thaats of Hindustani music, each with a seven-note interval set, tonal character, time-of-day association, and example ragas. **??** summarises the system.

The `thaat_for_scale()` function maps raga names used elsewhere in the engine (e.g., `"bhairav"`, `"todi"`) to their parent thaat, enabling the TUI to display tradition context alongside the active scale.

9.10 Rasa (Aesthetic Emotion)

The `theory/rasa` module implements the *navarasa* framework—nine canonical aesthetic emotions from Indian aesthetics —as complete synthesis parameter profiles. Each rasa maps to a tempo, preferred ragas, energy curve, filter character, reverb character, gamaka style, laya, nadai, drone volume, and shruti purity.

When a rasa is selected, its `RasaProfile` can be applied to `GeneratorConfig` to configure the entire synthesis chain from a single aesthetic intent.

Table 7. The 10 Hindustani thaats with interval sets and time associations.

Thaat	Intervals	Time / Character
Bilawal	[0, 2, 4, 5, 7, 9, 11]	Late morning / bright
Kalyan	[0, 2, 4, 6, 7, 9, 11]	Early evening / serene
Khamaj	[0, 2, 4, 5, 7, 9, 10]	Late evening / romantic
Bhairav	[0, 1, 4, 5, 7, 8, 11]	Dawn / devotional
Purvi	[0, 1, 4, 6, 7, 8, 11]	Evening twilight / serious
Marwa	[0, 1, 4, 6, 7, 9, 11]	Twilight / restless
Kafi	[0, 2, 3, 5, 7, 9, 10]	Late evening / romantic
Asavari	[0, 2, 3, 5, 7, 8, 10]	Late morning / pathos
Bhairavi	[0, 1, 3, 5, 7, 8, 10]	Any time / conclusion
Todi	[0, 1, 3, 6, 7, 8, 11]	Late morning / dark

Table 8. The nine rasas with representative synthesis parameters.

Rasa	Emotion	BPM	Energy	Filter	Laya	Shruti
Shringara	Love	125	wave	Warm	Ekgun	0.80
Karuna	Sorrow	118	yoga_flow	Dark	Vilambit	0.90
Raudra	Fury	150	tension_release	Aggressive	Chaugun	0.70
Veera	Heroism	145	full_set	Warm	Dugun	0.75
Bhayanaka	Terror	135	tension_release	Unstable	Tigun	0.80
Bibhatsa	Disgust	130	flat	Harsh	Ekgun	0.60
Adbhuta	Wonder	132	long_journey	Wide	Ekgun	0.85
Hasya	Joy	140	build_and_drop	Bright	Dugun	0.75
Shanta	Serenity	110	ambient_med.	Soft	Vilambit	1.00

9.11 Murchhana (Modal Rotation)

The `theory/murchhana` module implements scale rotation— the classical technique of deriving new ragas by starting a scale from a different degree. The algorithm subtracts the pivot degree's semitone value from all intervals modulo 12, then sorts and deduplicates:

$$I'_k = (I_k - I_d) \bmod 12, \quad k = 0, \ldots, 6 \tag{22}$$

The resulting interval set is cross-referenced against all known ragas and the 10 thaats to identify matches. This enables the engine to discover related ragas through transposition, supporting modulation planning and scale reinterpretation.

9.11.1 Live Murchhana Rotation

Murchhana is integrated into the render pipeline via the `murchhana_degree` field on `GeneratorConfig` (optional, 0–6). When set, an `EffectiveKey` abstraction wraps the rotated interval set and is threaded through all pitch generation:

- **Bass patterns** — the function `generate_bass_pattern_with_arp_effective()` generates bass notes normally, then quantises each pitch to the nearest effective pitch class by minimum semitone distance.
- **Melody, alankar, and pakad** — each generator's `generate_bar_with_effective()` method indexes scale degrees against the rotated intervals rather than the original scale.
- **Drone tuning** — the shruti box Sa frequency is shifted to the rotated degree's pitch class, keeping the drone consonant with the new modal center.

The TUI exposes murchhana via Shift+1–7 (select rotation degree) and Shift+0 (disable), with the current degree displayed in the status header. Because murchhana modifies the `EffectiveKey` rather than the underlying scale, the original raga identity is preserved in metadata while the audible pitch set transforms.

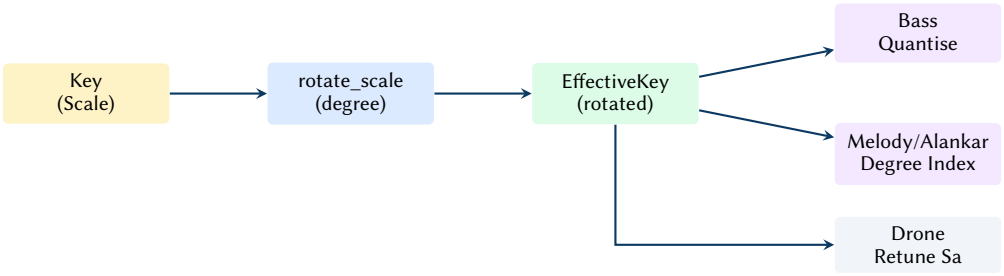

Figure 14. Murchhana integration: the original scale is rotated by degree, producing an EffectiveKey that is threaded through bass quantisation, melody generation, and drone retuning.

9.12 Chakra Energy Mapping

The `theory/chakra` module maps the macro energy level to a seven-chakra frequency model, where each chakra defines a spectral focus region, filter range, synthesis character, and element emphasis.

Table 9. Chakra-to-energy mapping. Each chakra defines the active spectral region as energy rises through the track.

Chakra	Energy Range	Filter (Hz)	Character
Muladhara	0.00–0.08	30–120	Deep, grounding
Svadhisthana	0.08–0.18	80–250	Flowing, warm
Manipura	0.18–0.35	150–500	Powerful, driving
Anahata	0.35–0.55	300–800	Open, melodic
Vishuddha	0.55–0.72	500–1200	Expressive, bright
Ajna	0.72–0.88	800–2000	Clear, piercing
Sahasrara	0.88–1.00	1500–8000	Ethereal, dissolving

9.13 Rhythmic Devices

The engine implements six Indian classical rhythmic devices as composable bar-level pattern transformations.

9.13.1 Tihai (Cadential Resolution)

The `TihaiGenerator` produces 16-step resolution patterns by expanding a 4-step phrase three times with 2-step gaps: $[P_4] + [G_2] + [P_4] + [G_2] + [P_4] = 16$ steps. Three phrase variants are available, chosen randomly. The triple repetition resolving on the final step creates a strong sense of arrival at sam (the downbeat).

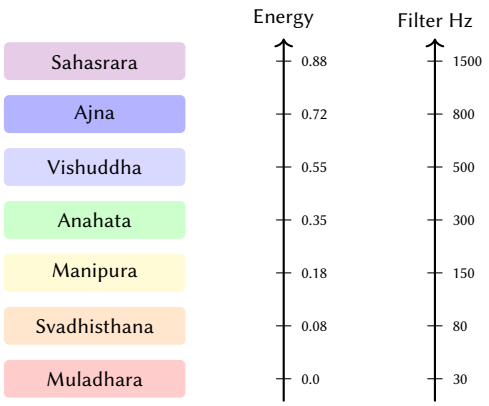

Figure 15. Chakra energy mapping: as macro energy rises, the active spectral focus ascends through the seven chakras.

9.13.2 Korvai (Acceleration Build)

The `KorvaiBuilder` implements an 8-bar progressive acceleration:

Table 10. Korvai acceleration stages over 8 bars.

Bars	Stage	Density
0–1	Quarter	4 hits/bar
2–3	Eighth	8 hits/bar
4–5	Sixteenth	16 hits/bar
6	Thirty-second feel	16 hits (accented)
7+	Resolve	1 hit (downbeat only)

9.13.3 Layakari (Time-Stretch)

The `LayakariState` rescales note positions within a bar by a multiplier drawn from five classical speed levels:

- **Vilambit** (0.5×) — half-time, meditative.
- **Ekgun** (1.0×) — normal speed.
- **Dugun** (2.0×) — double-time.
- **Tigun** (3.0×) — triple-time (polyrhythmic).
- **Chaugun** (4.0×) — quadruple-time.

The rescaling algorithm maps each note's step position: $s' = \text{round}(s/m)$, clamps to $[0, 15]$, and resolves collisions by retaining the highest-velocity note. Smooth transitions between speeds are interpolated over a configurable bar window.

Figure 16. Layakari time-stretching: the same rhythmic phrase at normal (1×), double (2×), and triple (3×) speed within a fixed 16-step bar.

9.13.4 Nadai (Subdivision Grouping)

The `NadaiState` imposes grouping-based accent patterns over the 16-step grid. Five subdivision sizes are supported: Chatusra (4), Tisra (3), Khanda (5), Misra (7), and Sankirna (9). The accent pattern assigns velocity multipliers:

$$v_k = \begin{cases} 1.0 & k \bmod g = 0 \quad \text{(primary accent)} \\ 0.6 & k \bmod g = 1 \quad \text{(secondary accent)} \\ 0.3 & \text{otherwise} \quad \text{(weak)} \end{cases} \tag{23}$$

where g is the group size. Nadai selection can be driven automatically by macro energy: Chatusra below 0.55, Tisra from 0.55 to 0.72, Khanda from 0.72 to 0.86, Misra from 0.86 to 0.95, and a return to Chatusra above 0.95 for the final climax.

9.13.5 Tani Avartanam (Solo Structure)

The `TaniAvartanam` defines a four-phase solo section (Opening, Development, Climax, Resolution) scheduled at a specific bar range. During a tani, percussion elements are suppressed and melodic density increases progressively, creating a breath in the accompaniment.

9.13.6 Sangati (Progressive Elaboration)

The sangati module adds a subtle energy boost that rises linearly over 8-bar cycles: $\Delta E = (b \bmod 8)/8 \cdot 0.12$, creating a "second time through" feel where each repetition of a phrase is slightly more energetic than the last.

9.14 Jugalbandi (Duet Framework)

The `JugalbandiMode` enum defines four call-and-response modes: Off, MelodicDuet (lead + pad/drone interaction), RhythmicDuet (kick + bass polyrhythmic conversation), and Full (all elements in duet mode). The active mode is controlled by the performance arc (**??**).

9.15 Performance Arc (Alap–Jor–Gat)

The `PerformanceArc` orchestrates the entire track trajectory using the classical three-part raga concert structure.

9.15.1 Phase Boundaries

The arc divides the total bar count into three phases using mode-dependent ratios:

Table 11. Performance arc mode boundaries as percentages of total bars.

Mode	Alap (%)	Jor (%)	Gat (%)
Standard	30	30	40
AlapHeavy	40	25	35
GatHeavy	20	20	60

9.15.2 Feature Gating

Each phase activates a `FeatureSet` that controls which synthesis elements, rhythmic devices, and ornamental techniques are available:

- **Alap** — drone and lead only. Breath LFO active, gamaka intensity 1.0, no percussion. Solo melodic exploration.
- **Jor** — kick and bass enter. Jugalbandi set to MelodicDuet, sangati enabled, gamaka intensity 0.7.

- **Gat** — all elements active. Korvai, tihai, tani, and layakari enabled. Jugalbandi escalates from MelodicDuet (first 70%) to Full (final 30%). Jhala activates in the final 15%. Gamaka intensity decreases from 0.55 to 0.35 as rhythmic clarity takes priority.

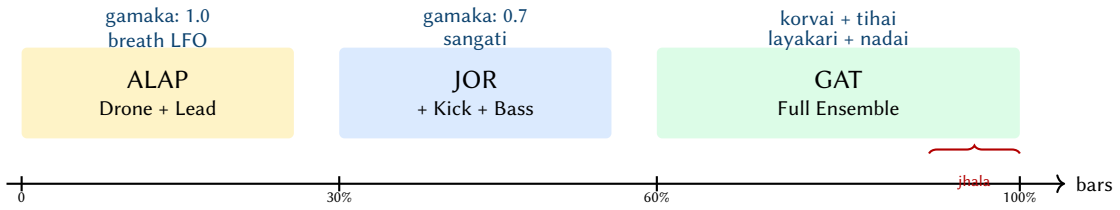

Figure 17. Performance arc: the classical Alap–Jor–Gat structure progressively activates synthesis elements and rhythmic devices. Gamaka intensity decreases as rhythmic complexity increases.

9.16 Planned Extensions

The same architecture supports additional eastern concepts as future work: extended murchhana-based modulation planning, a Vilambit– Madhya–Drut tempo layering within the Gat phase, and deeper rasa-to-parameter mapping for emotion-driven composition.

9.17 Implemented Music-Theory Features (Eastern Layer)

The eastern layer is not solely conceptual: it is implemented across the generator configuration, theory primitives, and TUI interaction model. This section documents the concrete features that currently ship, how they are represented in code, and how they influence rendering.

Raga Scale Sets and Phrase Seeds Raga-coloured scale sets are defined in the `theory` layer as interval lists (12-TET semitone offsets) with optional phrase metadata in `src/theory/scales.rs`. In addition to a scale *name* and *intervals*, selected ragas expose a *pakad* structure: a characteristic phrase plus explicit ascending/descending degree sequences. This enables two distinct uses:

- **Constraint** — all pitch selection is constrained to the active scale.
- **Identity injection** — pakad phrases can be injected as melodic seeds, giving a recognisable contour beyond raw interval choice.

Pakad Engine (Phrase-Based Melody Generation) When a selected scale provides a pakad, the engine can instantiate a `PakadEngine` (`src/engine/pakad.rs`) that cycles through a small phrase bank (characteristic, ascending, descending). For each bar it generates a sparse-to-dense pattern as a function of macro energy:

- Density increases with energy (fewer notes at low energy; more notes at high energy).
- Step stride shortens as energy rises (quarter-note feel → 8ths → 16ths).
- A controlled variation mode perturbs degrees by ±1 with probability increasing above mid energy, while remaining in-scale.

This mechanism is intentionally lightweight: it preserves the generator's parameterised design while allowing raga identity to emerge from recurring phrase contours.

Alankar Generator (Permutation Exercises as Ornamented Runs) The generator includes an `AlankarGenerator` (`src/engine/alankar.rs`) that emits bar-length note patterns derived from alankar-style degree permutations. Implemented variants include Sequential, Skip, Mirror, Spiral, Zigzag,

and Vakra (a constrained random walk). For instance, the *Sequential* permutation generates degrees by sliding a window:

$$D_i = (C + i) \bmod N, \quad i = 0, \ldots, M - 1 \tag{24}$$

where C is the current scale cursor, N is the number of intervals in the scale, and M is the number of notes generated per bar based on the laya (speed) and energy level. The generator exposes:

- **Type** — selects the permutation strategy.
- **Window size** — controls the size of the scale-degree window used by some strategies.
- **Direction** — ascending, descending, or stochastic.
- **Speed (laya)** — maps to steps-per-note (e.g., slower at low energy; faster at high energy).

In the current composition logic the alankar layer is treated as a high-energy embellishment: it can take over lead generation above an energy threshold to produce fast, raga-constrained runs without requiring explicit note programming.

Bol Text Input (Tabla Syllables to Step Grid) The TUI provides a bol text entry mode for quickly imposing a tala/tabla-inspired rhythm onto the tom/percussion lane (`src/tui/app.rs`). A bol string is parsed by the bol parser (`src/engine/bol_parser.rs`) into tokens; each token can carry an articulation label and a velocity. Tokens are then mapped onto a 16-step bar by wrapping or truncation. The resulting per-step overrides are written into the next bar's pattern override state (steps, velocity, and articulation arrays) via the step-edit override structures in `src/engine/generator.rs`.

This feature provides a human-readable rhythmic interface: rather than editing 16 booleans, a user can type a mnemonic sequence and immediately audition it in context.

Shruti Purity (Microtonal Blend) Microtonal behaviour is exposed as a continuous `shruti_purity` parameter in the generator configuration (`src/engine/generator.rs`).

- **0.0** — strict 12-TET tuning.
- **1.0** — apply the full raga-specific shruti offset table (when available for the active scale).

At render time, note frequencies are derived via a tuning function *after* degree selection: the melodic logic remains scale-degree based, while the final oscillator frequencies can slide microtonally. This separation keeps pattern generation deterministic and allows shruti to be treated as a timbral/intonational layer.

Raga Modulation (Scale Transition Over Bars) The engine supports raga modulation as a scheduled transition from a source raga to a target raga over B_{trans} bars beginning at B_{start} (`src/engine/raga_modulation.rs`). The modulation state evaluates a linear progress function t for the current bar b:

$$t(b) = \text{clamp}\left(\frac{b - B_{\text{start}}}{B_{\text{trans}}},\ 0,\ 1\right) \tag{25}$$

In practice, modulation is implemented as scale blending: common tones are favoured early in the transition, while notes unique to the target scale are introduced as $t(b)$ increases. This enables mid-track colour changes without abrupt key changes or DAW-side automation.

Time-of-Day (Samay) Mood Selection The `samay` mood preset (`src/theory/mood.rs`) selects a raga and an energy curve based on local time-of-day (hour buckets). This couples two orthogonal parameters:

- **Scale choice** — biases toward ragas traditionally associated with certain times.
- **Energy architecture** — selects among curves such as `yoga_flow`, `long_journey`, and `ambient_meditation` to match the intended affect.

This feature demonstrates the intended design philosophy: music theory is encoded as deterministic selection logic over user-facing parameters, not as opaque learned behaviour.

Breath/Chakra/Arc Controls (Theory-Inspired Macro Behaviour) Several additional switches connect theory-inspired concepts to synthesis and arrangement:

- **Breath-synced LFO** — a slow modulation source that can shape brightness and wetness, particularly effective for low-energy outputs.
- **Chakra mode** — a global mode that maps sections/energy to chakra-derived colour and parameter bias.
- **Performance arc** — an Alap/Jor/Gat-inspired gating mode that controls feature activation over time.

These controls do not replace the energy model; rather, they provide interpretable, named presets that guide how energy maps onto timbre and pattern density.

User-Facing Controls in the TUI The eastern layer is directly manipulable in the terminal UI (`src/tui/app.rs` and `src/tui/ui.rs`). Notable bindings include:

- **Bol entry** — **:** enters bol input mode; on submit the bol string is applied to the next bar's tom lane.
- **Raga modulation** — a shifted key binding cycles through eastern ragas and schedules a transition over a fixed bar window.
- **Alankar** — a shifted key binding cycles the alankar type (and enables it); an alternate-modifier binding toggles alankar on/off.
- **Shruti purity** — incremental adjustments blend between 12-TET and shruti offsets.
- **Murchhana** — Shift+1 through Shift+7 selects a rotation degree; Shift+0 disables. The status header shows the active degree.
- **Samay** — a dedicated binding applies the time-of-day recommendation (mood preset swap).

Because the generator treats the configuration as the single source of truth (**??**), each toggle results in a deterministic re-render that updates both the audio output and the visible state.

Feature-to-Implementation Map **??** summarises the primary entry points for each implemented feature: the configuration field that carries it, the module that implements it, and the principal render-time effect.

Configuration Surface (Eastern Parameters) The eastern layer is exposed as explicit fields on `GeneratorConfig` (`src/engine/generator.rs`). This makes the feature set accessible from the CLI, from preset serialization (`serde`), and from the TUI re-render loop.

Table 13. Eastern-layer configuration fields in `GeneratorConfig`.

Field	Type	Meaning / Notes
`mood_name`	`String`	Selects a mood preset (e.g., `"eastern"`, `"meditative"`, `"devotional"`, `"samay"`); mood determines key/scale, energy curve name, and filter ranges.
`key_override`	`Option<(u8,String)>`	Optional override of the mood's root MIDI note and scale name (e.g., `(38, "bhairav")`); applied when resolving the mood.
`energy_curve_override`	`Option<String>`	Overrides the macro curve preset name (e.g., `"yoga_flow"`, `"long_journey"`, `"ambient_meditation"`); used by `MacroCurve::from_name`.
`drone_enabled`	`bool`	Enables tanpura-style drone synthesis layer; typically mixed inversely with macro energy to support sparse/meditative sections.
`breath_lfo_enabled`	`bool`	Enables breath-synced modulation (slow LFO) to shape brightness/wetness; useful for yoga/ambient curves.
`chakra_mode`	`bool`	Enables chakra-derived parameter bias and/or visual mapping in the TUI; can also be implied by yoga-style energy presets.
`shruti_purity`	`f64`	Microtonal blend in $[0, 1]$: `0.0` = 12-TET; `1.0` = full shruti offsets (when the active scale provides a tuning map).
`performance_arc_mode`	`Option<ArcMode>`	Enables Alap/Jor/Gat-inspired gating of features over time; influences when melodic density and layers become available.
`modulation`	`Option<RagaModulationConfig>`	Scheduled source→target raga transition: `start_bar`, `transition_bars`, and progress function t used at render time for scale blending.
`alankar_enabled`	`bool`	Enables alankar-derived melodic generation as a lead-mode enhancement (typically above an energy threshold).
`alankar`	`AlankarGeneratorConfig`	Alankar configuration (type, window size, direction, speed/laya, cursor state); serialized so repeated renders continue the pattern evolution.

Continued on next page

Field	Type	Meaning / Notes
`murchhana_degree`	`Option<u8>`	Murchhana rotation degree (0–6); when set, an `EffectiveKey` wraps the rotated interval set for bass quantisation, melody/alankar/pakad degree indexing, and drone retuning. TUI: Shift+1–7 to set, Shift+0 to disable.
`tani_trigger_bar`	`Option<u32>`	Schedules a percussion-solo style event (tani avartanam) beginning at a bar index; used as an arrangement accent.
`jugalbandi_mode`	`JugalbandiMode`	Selects call-and-response behaviour between voices; interacts with melody/pattern generation rather than tuning directly.
`layakari_enabled`	`bool`	Enables rhythmic multiplication/time-warp within bars (ekgun→dugun→tigun...); affects event timing density.
`layakari`	`LayakariState`	Carries the current/target laya and transition scheduling; interpreted per bar at render time.
`nadai_enabled`	`bool`	Enables subdivision-group accent warp (tisra/khanda/misra...); applied as a velocity emphasis pattern over 16 steps.
`nadai`	`NadaiState`	Holds the current nadai selection and related transition state (if any).

The remaining configuration fields (filter/DSP, sample placement, pattern presets, per-bar overrides) are shared across western and eastern modes; the eastern layer composes with these rather than replacing them.

9.18 Music Theory Primer for Parameterised Generation

The generator exposes musical concepts as user-facing parameters (key, scale, mood, energy curve) rather than as low-level note events. Consequently, a compact, implementation-oriented music theory vocabulary is useful to explain how scale choices, rhythmic emphasis, and harmonic density interact with the engine.

Pitch, Key, and Scales All pitched content is derived from a root pitch class (key) and a scale defined as semitone offsets within the octave. A scale thus becomes a constraint function: it filters candidate MIDI pitches (or oscillator frequencies) down to a musically coherent subset.

In Western terms, common scale families include major (Ionian), natural minor (Aeolian), and modal rotations (Dorian, Phrygian, Mixolydian). In eastern terms, the project also includes raga-coloured interval sets that emphasise characteristic seconds and sixths. Because the engine operates primarily in 12-TET, these are approximations; microtonal extensions (shruti offsets) are treated as future work (**??**).

Degrees, Stability, and Nyasa The system models pitch selection primarily in terms of scale degrees. This is compatible with both modal western usage and raga-based melodic grammar. A practical implementation benefit is that stability can be encoded as a simple weighting curve over degrees: at low energy the engine prefers stable tones (root, fifth) and nyasa-like landings; at higher energy it introduces more colour tones (second, fourth, sixth) and wider register.

In the raga-oriented features, *nyasa* is treated as a cadence target instead of a strict rule system: the harmony layer can hold a nyasa tone for a bar (`src/engine/harmony.rs`), and melodic generators can bias phrase endings toward vadi/samvadi when they are annotated on the active scale.

Harmony and Implied Progression Psytrance often relies on implied harmony rather than functional chord progressions: bass + lead motion can suggest a tonal center even when no triads are voiced. Within the engine, harmony is therefore modelled as a lightweight layer:

- A scale constrains pitch selection.
- A degree-weighting profile biases landings toward stable tones (root, fifth) at low energy and toward colour tones (second, fourth, sixth) at higher energy.
- Energy can modulate harmonic density by enabling additional voices or widening pitch ranges.

This approach is compatible with both Western modes and raga-inspired sets, and avoids over-specifying chord symbols that are uncommon in the genre.

Rhythm, Meter, and Accent The system renders onto a fixed 4/4 transport grid with 16-step subdivision, but rhythm is perceived primarily through accent patterns. Classic EDM emphasis (beats 1 and 3) is augmented by syncopation styles and by tala-derived accent cycles (**??**). As a result, the engine can create the illusion of longer cycles (e.g., 7 or 10) while remaining compatible with DAW timelines and MIDI export.

Mode Rotation (Murchhana / Graha Bhedam) Beyond choosing a scale, the generator supports deriving *related* scales via mode rotation: selecting a new starting degree (graha) and re-normalising the resulting interval set to a zero-root representation. This is implemented in `src/theory/murchhana.rs` and is applied via an `EffectiveKey` abstraction so the rest of the engine can remain degree-based.

Musically, murchhana provides a controlled way to evolve harmonic colour over time while keeping the rhythmic surface stable. It can be applied as a section boundary device (e.g., at bar 17) or as a longer journey in combination with raga modulation.

Form and Energy Arrangement is expressed as an energy function over bars. Macro energy selects sections (intro, build, peak, breakdown), meso energy creates breathing, and micro energy controls per-hit density. Music theory concepts connect directly to this model:

- Lower energy favours fewer notes, narrower ranges, and stronger tonic anchoring.
- Higher energy supports faster note rates, wider ranges, and more dissonant colour tones.
- Cadences (e.g., tihai-like resolutions) provide audible section boundaries even without explicit chord changes.

10 Ableton Integration

The generator integrates with Ableton Live through three complementary mechanisms: MIDI file export, real-time OSC clip pushing, and Ableton Link transport synchronisation.

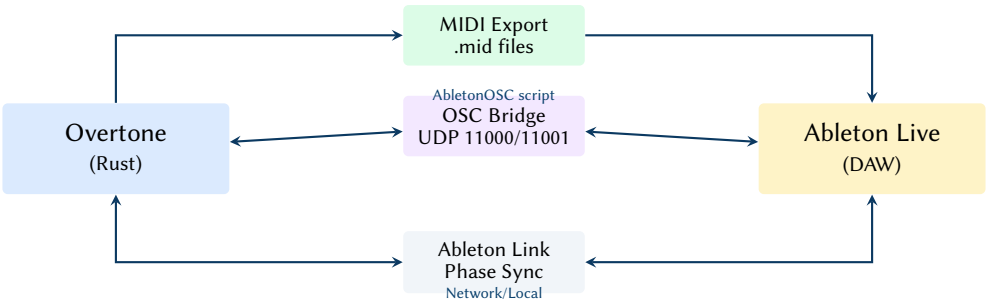

Figure 18. Ableton Live integration architecture. Overtone communicates via Standard MIDI Files for static export, bidirectional OSC for clip pushing and scene control, and Ableton Link for phase-locked transport synchronisation.

10.1 MIDI Export

The `midi` module generates Standard MIDI Files (SMF Type 1, 960 PPQ) using the `midly` crate. Each non-muted element is written as a separate track. Drum elements use General MIDI standard note numbers (kick = 36/C1, closed hi-hat = 42/F#1, clap = 39/D#1). Bass notes are converted from frequency to MIDI pitch via $p = 69 + 12 \log_2(f/440)$.

10.2 MIDI Import

The `midi_import` module parses Standard MIDI Files and classifies tracks into generator lanes (Kick, Bass, Hihat, Clap, Other) using a two-stage strategy:

1. **Track name matching** — case-insensitive keyword search for "kick", "bass", "hihat"/"hat", "clap"/"snare" in the MIDI track name meta event.
2. **General MIDI drum note** — fallback to GM note numbers: 36 → Kick, 42/46 → Hihat, 38–40 → Clap.

The user can select the classification mode: TrackNameThenGm (default, tries names first), TrackNameOnly, or GmOnly. The parser maintains a `HashMap<(channel, key), (start_tick, vel)>` of active notes, closing them on NoteOff or zero-velocity NoteOn. Imported notes are sorted per lane by tick and stored in an `ImportedMidiSnapshot` alongside the session for recall.

10.3 OSC Bridge

The `osc` module provides a bidirectional UDP bridge to Ableton Live via the AbletonOSC remote script [**abletonosc**], which exposes Live's Python API over OSC on ports 11000 (send) and 11001 (receive).

10.3.1 Architecture

The OSC subsystem consists of three components:

- `OscClient` — a `UdpSocket` wrapper that encodes and sends OSC messages using the `rosc` crate.
- `start_listener` — a background thread that binds to port 11001 and decodes incoming OSC responses, forwarding them through an `mpsc` channel.
- `AbletonBridge` — a high-level command API for clip management (`create_clip`, `add_notes`, `fire_clip`), scene control (`create_scene`, `fire_scene`), and transport (`start_playing`, `stop_playing`).

10.3.2 Clip Push Workflow

When the user triggers a clip push (or when a re-render occurs with an active OSC connection), the bridge executes the following sequence per element:

1. Delete existing clip in slot 0 of the target track.

2. Wait 20 ms (Ableton requires a brief pause after clip deletion).
3. Create a new clip of the appropriate length in beats.
4. Set the clip name to `Element – MoodName`.
5. Add all note events with pitch, start beat, duration, and velocity.

Track assignment is resolved by case-insensitive matching of Ableton track names against the element names ("kick", "bass", "hihat", "clap").

10.4 Ableton Link

The `link` module wraps the `rusty_link` crate [**rustylink**]—a Rust FFI binding to Ableton's C++ Link library—to provide tempo and phase synchronisation with any Link peer (Ableton Live, other Link-enabled applications, or hardware).

10.4.1 Tempo Synchronisation

The `LinkBridge` registers callbacks via `set_tempo_callback` and `set_num_peers_callback` that forward events through an `mpsc` channel, drained by the TUI event loop each frame. Tempo changes from any peer propagate within one beat. A feedback-loop guard ignores tempo deltas smaller than 0.001 BPM.

10.4.2 Phase-Locked Playback

When Link peers are present, the generator schedules playback start at the next bar boundary (quantum = 4 beats). The `cpal` audio callback maps Link beat positions to buffer frame indices using the `HostTimeFilter`:

$$\text{pos}_{\text{frames}} = \text{offset}_{\text{start}} + (b_{\text{link}} - b_{\text{origin}}) \cdot \frac{f_s \cdot 60}{\text{BPM}} \tag{26}$$

where b_{link} is the current Link beat position and b_{origin} is the beat at which playback was initiated.

10.5 Scene Control

The energy model's arrangement sections (Intro, Build, Peak, Breakdown) are mapped to Ableton scenes. When `follow_energy` mode is enabled, the generator fires the appropriate scene automatically as the bar cursor advances through section boundaries, driven by beat callbacks from the OSC listener.

11 Terminal User Interface

The TUI is implemented with `ratatui` [**ratatui**] and `crossterm`, running at a 50 ms frame cadence. It provides a state machine with six screens: Launch, Playback, StepEdit, SessionList, PresetList, and EnergyEditor.

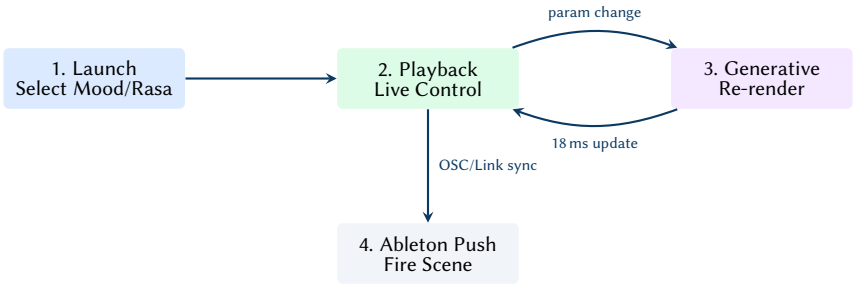

Figure 19. The user workflow journey. Selecting a mood configures the global state. Adjustments during playback trigger an instantaneous generative re-render, with structural updates pushed seamlessly to Ableton Live.

11.1 Screen State Machine

- **Launch** — parameter configuration: mood selection, BPM/bars adjustment, theme browsing, key/scale overrides, filter/LFO/arp settings, sample directory selection.
- **Playback** — live visualisation of pattern grids (4-bar view with per-step markers), energy curve sparkline, waveform display, element mute/solo/density controls, OSC/Link status indicators, and real-time parameter knobs.
- **StepEdit** — per-bar step editing with toggle-able 16th-note grid cells. Overridden patterns bypass the generative algorithm for the edited bar.
- **SessionList / PresetList** — browsable lists with timestamps and metadata, supporting load, save, and delete operations.
- **EnergyEditor** — direct manipulation of macro curve control points.

11.2 Re-Render Protocol

Parameter changes trigger a full re-render: the TUI stops playback (recording the current bar position), constructs a new `Generator`, calls `render()`, and resumes playback from the same bar offset. At 18 ms per render (**??**), this produces an imperceptible gap in audio output.

11.3 Theme System

Twelve theme presets are defined as a static array, each bundling a mood, BPM, key, scale, energy curve, and DSP parameters into a coherent sonic identity. Themes range from classic psytrance (Dark Phrygian, Acid Machine) through uplifting variants (Euphoric Rise, Solar Flare) to experimental settings (Alien Signal, Hitech Assault). Applying a theme overwrites all relevant fields in the `GeneratorConfig` via `apply_to()`.

11.4 Tala Circle Visualisation

The `tala_circle` module renders a circular beat indicator in the TUI, displaying the current tala cycle as a ring of glyphs. Each beat position is mapped to polar coordinates ($\theta = 2\pi \cdot b/B - \pi/2$, starting from top-centre) and rendered with type-specific glyphs:

- **Sam** (downbeat) — red filled circle, yellow arrow when active.
- **Khali** (empty beat) — blue double circle.
- **Normal** — grey open circle.

The visualisation respects the tala's vibhag structure (groupings of beats with assigned accent types), providing a musically meaningful display of rhythmic position within the cycle.

11.5 Energy Colour Mapping

The TUI uses a four-tier colour gradient to represent energy levels throughout the interface (pattern grid cells, energy sparkline, status indicators):

12 Streaming Engine

The batch pre-render pipeline described in **??** produces a complete track in a single pass. The `streaming` module provides an alternative: a sample-accurate, event-driven engine that processes audio in blocks, enabling real-time playback with lower latency and incremental state updates.

12.1 Architecture

The `StreamingEngine` pre-sorts all note events by absolute sample position at construction time, then processes them via a linear cursor scan during block rendering. The engine maintains per-element synth

instances (kick, bass, hi-hat, clap, lead, pad, tom, drone), an effects chain (sidechain compressor, ping-pong delay, Schroeder reverb, master HPF), and an optional breath LFO for ambient modulation.

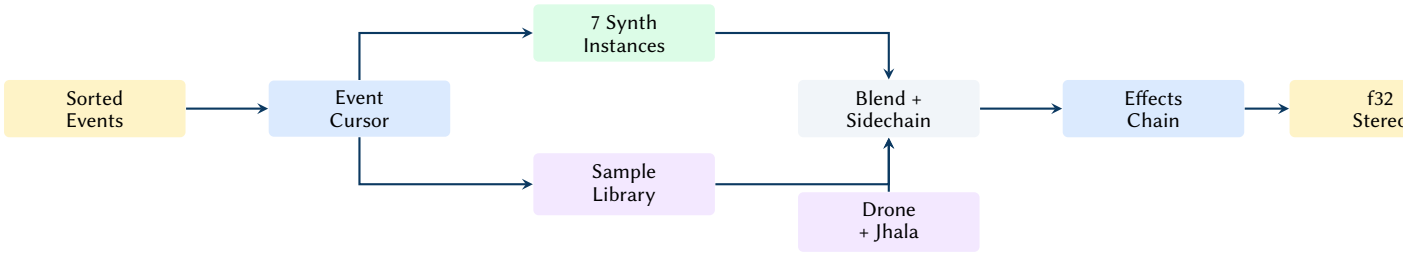

Figure 20. Streaming engine architecture: pre-sorted events are dispatched sample-accurately, blended with energy-aware sample playback, and processed through the effects chain.

12.2 Block Processing

For each sample frame in a block, the engine:

1. Computes the current bar index from the global sample clock and triggers bar-level sample placements (FX, risers, textures, vocals) at bar boundaries.
2. Dispatches all scheduled events at the current sample position, triggering the appropriate synth with energy-aware blend ratios ($r = \min(0.8 \cdot E_{\text{macro}}, 0.8)$).
3. Renders all active synth voices and sample playback instances, applying sidechain compression to the bass.
4. Optionally modulates volume (±15%), filter cutoff (±30%), and effects wet level (±20%) via the breath LFO.
5. Adds the drone and jhala layers (stereo).
6. Processes the stereo bus through the delay, reverb, and master HPF chain.
7. Outputs interleaved `f32` stereo samples.

12.3 Synth/Sample Blending

Each element's blend ratio is computed from the macro energy at the current bar. The blending strategy varies by element:

- **Kick** — synth velocity scaled by $(1 - r_{\text{blend}})$; sample played at full velocity.
- **Bass** — at blend ≥ 0.8, sample-only mode (synth bypassed entirely).
- **Hi-hat/Clap** — at blend ≥ 0.5, sample preferred; synth used as fallback.
- **Lead** — pure synthesis (no sample blending).
- **Pad** — voiced as a minor chord (root, +3, +7, +10 semitones).

12.4 Sample Library

The `SampleLibrary` indexes loaded samples by (SampleCategory, EnergyZone) pairs, with round-robin counters per pair to prevent repetition. Samples are classified into nine categories (Kick, Snare, Hat, Clap, Bass, FX, Riser, Texture, Vocal) and three energy zones (Low, Mid, High). Zone assignment uses RMS level thresholds: Low below 0.08, Mid between 0.08 and 0.22, High above 0.22. For energy-aware selection during playback, zones are alternatively assigned from the macro energy curve: Low below 0.35, Mid between 0.35 and 0.70, High above 0.70.

When a category has samples in only one energy zone, the library automatically redistributes them into Low, Mid, and High zones by sorting on RMS level and dividing into equal thirds. Queries fall back through zones in priority order (requested → Mid → remaining) to maximise sample utilisation.

12.5 Playback and Live Parameters

The `PlaybackHandle` wraps a `cpal` audio stream with lock-free parameter control via `AtomicU64` (f64 bit patterns) and `AtomicUsize` for filter type. Four live parameters are exposed: cutoff frequency, resonance, gain, and filter type.

The audio callback checks for parameter changes each block using threshold guards ($|\Delta f_c| > 0.1$ Hz or $|\Delta Q| > 0.01$) and recomputes biquad coefficients only when needed. Two playback modes are supported:

- **Buffer playback** — pre-rendered samples are read sequentially with per-sample live filtering and gain.
- **Streaming playback** — the `StreamingEngine` is locked per block, live parameters are injected into its config, and `process_block()` renders directly into the output buffer.

When Ableton Link is active, the buffer playback mode uses per-sample beat position interpolation:

$$p_{\text{frames}} = p_{\text{start}} + (b_{\text{link}} - b_{\text{origin}}) \cdot \frac{f_s \cdot 60}{\text{BPM}} \tag{27}$$

with linear sample interpolation between adjacent frames to avoid clicks at fractional positions.

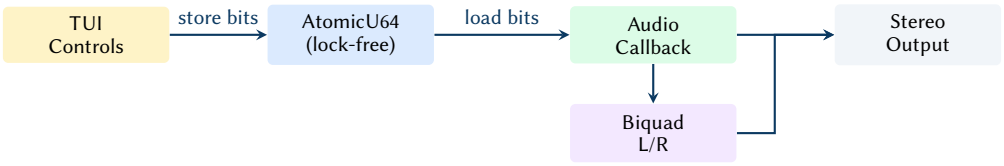

Figure 21. Lock-free live parameter path: the TUI stores f64 bit patterns into atomics; the audio callback loads them each block and updates the biquad filter only when thresholds are exceeded.

13 Session and Preset Management

13.1 Sessions

A `Session` captures the complete generator state (name, UTC timestamp, full `GeneratorConfig`, and an optional `ImportedMidiSnapshot`) as a JSON file. Sessions are named `{name}_{YYYYMMDD_HHMMSS}.json` and stored in a user-specified directory. The `list_sessions()` function scans the directory and returns all sessions sorted newest-first.

13.2 Presets

A `Preset` snapshots the audio-relevant subset of the generator configuration—mood, energy curve, filter settings, delay/reverb wet levels, pattern presets for all elements, sample blend overrides, and humanisation level—tagged with a name, timestamp, and search tags. Unlike sessions, presets omit runtime state (mute/solo, playback position) and focus on reproducible sound design.

The `from_config()` constructor extracts preset parameters from a live `GeneratorConfig`, and `apply_to()` writes them back, enabling rapid A/B comparison and preset morphing workflows.

14 Evaluation

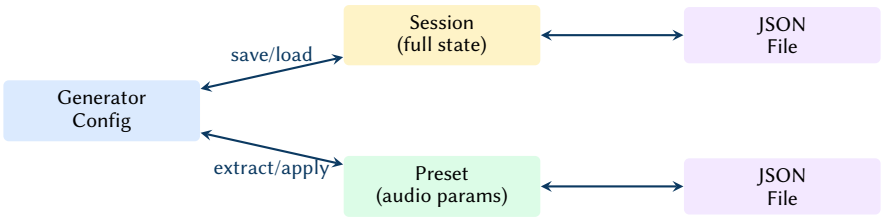

Figure 22. Session and preset persistence: sessions capture full state, presets capture audio parameters only. Both serialise to JSON.

We evaluate the generator on two axes: rendering throughput and output quality characteristics. All benchmarks were performed on a workstation with an Apple M2 processor and 16 GB of RAM, running Rust 1.82 on macOS 15.

14.1 Rendering Performance

?? reports rendering times for tracks of increasing length. All renders use the *dark* mood preset at 142 BPM with default DSP parameters and no sample blending (pure synthesis).

The sub-linear scaling (real-time ratio *improves* with track length) reflects the fixed overhead of energy model initialisation and DSP state allocation, which is amortised over longer renders.

> **Sample Blending Overhead**
>
> When sample blending is active, render times increase by approximately 15–25% due to the additional PCM resampling and gain staging. A 64-bar track with full sample blending renders in approximately 22 ms—still well within the 50 ms TUI frame budget.

14.2 Memory Footprint

The system maintains a low memory profile. The core engine and DSP state allocate statically, requiring less than 50 MB. The `SampleLibrary` manages memory by eagerly extracting `rustfft` profiles during directory scans but caching them in sidecar JSON files (`*.sample.json`). During playback, only the necessary `f32` audio buffers for scheduled samples are loaded into RAM, keeping the footprint tightly bounded even with extensive sample directories.

14.3 Concurrency Analysis

Real-time audio demands non-blocking, wait-free coordination between the TUI thread and the `cpal` audio callback. The engine achieves this via `AtomicU64` types that store `f64` bit patterns (`f64::to_bits()`). When the user adjusts a live parameter (e.g., filter cutoff) in the TUI, it is written atomically. The audio block processor loads these values continuously without acquiring mutexes. This lock-free design ensures the audio thread is never preempted by UI updates, avoiding xruns (buffer underruns) and ensuring a glitch-free 50 ms re-render cadence.

14.4 Signal Characteristics

The generator produces output with the following signal properties at default settings:

- **Peak level**: −0.5 dBFS (after normalisation and limiting).
- **RMS level**: approximately −12 to −8 dBFS during peak energy sections, consistent with mastered electronic music.
- **Frequency range**: 30 Hz to 20 kHz, bounded by the master HPF and the Nyquist frequency.
- **Stereo field**: kick and bass centre-panned; hi-hat slightly right (pan 0.55); clap slightly left (pan 0.45).

- **Internal precision**: 64-bit floating-point throughout the signal path; 16-bit integer at WAV export.

15 Conclusion

This report has presented Overtone, an energy-driven cross-cultural generative music engine that produces full arrangements from declarative parameter configurations. The key contribution is the four-level energy model (??) that replaces manual arrangement with a hierarchical abstraction: macro curves define the track arc, meso patterns add phrase-level breathing, micro grooves shape per-step accents, and sub-channel gating controls per-element entry and response characteristics.

The synthesis pipeline (??) implements nine dedicated engines—kick, bass, hi-hat, clap, FM lead, tanpura drone, ensemble pad, Karplus–Strong plucked strings (sitar, veena, sarod, santoor), shruti box, and tabla-articulated tom—all operating at 64-bit precision with per-sample filter modulation. The DSP layer (??) provides wavetable morphing with bilinear cross-cycle interpolation and PolyBLEP anti-aliased oscillators alongside the biquad filters, Schroeder reverb, and ping-pong delay.

The sample intelligence subsystem (??) extends the sonic palette by analysing, classifying, and blending audio samples with synthesised output based on the energy state. The sample library (??) indexes samples by category and energy zone with automatic redistribution and round-robin selection.

The eastern musical traditions layer (??) implements the 72-melakarta Carnatic classification, the 10 Hindustani thaats, the navarasa aesthetic framework, and murchhana modal rotation for scale discovery. Six classical rhythmic devices—tihai, korvai, layakari, nadai, tani avartanam, and sangati—compose within an Alap–Jor–Gat performance arc that progressively activates synthesis elements and ornamental techniques. Raga-aware melody generation respects aroh/avroh paths and assigns per-note gamaka ornaments (andolan, kampita, meend, kan-swar).

The Ableton integration (??) bridges the gap between generative composition and professional production, enabling MIDI export and import, real-time OSC clip pushing, Link transport synchronisation, and energy-driven scene control. Session and preset management (??) provides full-state persistence and reproducible sound design snapshots.

The streaming engine (??) provides sample-accurate, event-driven block processing as an alternative to the batch pre-render model, with energy-aware synth/sample blending and breath LFO modulation.

Rendering performance (??) exceeds 5,000× real time for typical track lengths, enabling the TUI's re-render-on-change workflow where any parameter adjustment produces an updated mix in under 20 ms.

Future development directions include moving toward a comprehensive spatial audio model, integrating granular and spectral-aware synthesis for Pad and Atmosphere generators, and expanding physical modelling to transcend the rigid Karplus-Strong string and hardcoded percussive envelopes. Furthermore, completing the DAW integration workflows with robust MIDI import overriding will close the loop between generative discovery and manual editing.

Critical Assessment

Despite these achievements, the current architecture harbours significant technical debt and rigid design biases. The reliance on 64-bit precision and per-sample coefficient updates wastes computational headroom that could be better spent on oversampling and anti-aliasing—crucial omissions that degrade the high-frequency fidelity of the FM lead and basic oscillators. Furthermore, the synthesiser implementations are heavily reliant on hardcoded parameters (e.g., fixed ADSR slopes, rigid clap multi-taps, exact

0.3s kick durations). This lack of parameterization locks the engine into its primary psytrance use case, directly conflicting with its goal of broad cross-cultural and cross-genre flexibility. Addressing these limitations through modular parameterization and proper bandlimiting is essential before the engine can be considered a truly general-purpose generative tool.

Table 12. Implemented eastern layer features: config fields, implementation sites, and effects.

Feature	Config / Entry Point	Implementation + Effect
Raga scales + pakad	`Key.scale` (mood / override)	`src/theory/scales.rs`: interval set + optional pakad phrases constrain pitch selection and provide phrase seeds.
Pakad melody	implicit via active scale pakad	`src/engine/pakad.rs`: phrase-bank bar generation with energy-scaled density/stride/variation.
Alankar runs	`alankar_enabled`, `alankar`	`src/engine/alankar.rs`: degree permutations emit fast raga-constrained runs; used as a high-energy lead mode.
Bol text input	TUI override workflow	`src/engine/bol_parser.rs` + `src/tui/app.rs`: parse bols to (step, vel, articulation) and write tom overrides for the next bar.
Shruti tuning	`shruti_purity`	`src/engine/generator.rs` + tuning functions: blend 12-TET to shruti offsets when available; applied at frequency computation.
Raga modulation	`modulation`	`src/engine/raga_modulation.rs`: schedule source→target transition; render-time scale blending introduces target-unique tones over bars.
Murchhana	`murchhana_degree`	`src/theory/murchhana.rs` + `src/engine/effective_key.rs`: live modal rotation; bass quantised, melody/alankar/pakad indexed, drone retuned to rotated degree. TUI: Shift+1–7/0.
Samay mood	`mood_name="samay"`	`src/theory/mood.rs`: time-of-day raga + energy curve selection; presets filter ranges accordingly.
Yoga/ambient curves	`energy_curve_override` / mood preset	`src/engine/energy.rs`: named macro curves shape arrangement density and timbral evolution.
Breath/chakra/arc	`breath_lfo_enabled`, `chakra_mode`, `performance_arc_mode`	`src/dsp/lfo.rs` + `src/theory/chakra.rs` + arc logic: interpretable macro modulation and feature gating layered atop energy.

Table 14. Energy-to-colour mapping in the TUI.

Energy Range	Colour	Meaning
< 0.33	Blue	Low energy (intro, breakdown)
0.33–0.66	Green	Moderate energy (build)
0.66–0.85	Yellow	High energy (drive)
≥ 0.85	Red	Peak energy (climax)

Table 15. Rendering performance by track length (pure synthesis, 44.1 kHz stereo). The real-time ratio is computed as track duration / render time.

Bars	Duration (s)	Render Time (ms)	Real-Time Ratio
8	13.5	3	4,500×
16	27.0	5	5,400×
32	54.1	10	5,410×
64	108.2	18	6,011×
128	216.3	35	6,180×
256	432.7	68	6,363×